



Membri del team di sviluppo:

Luca Cimino 970880

Federico Mingarelli 998232

Stefano Spadari 970191

Sommario

ABSTRACT	3
ANALISI DEI REQUISITI	4
Raccolta dei requisiti	4
ANALISI DEL DOMINIO	9
ANALISI DEI REQUISITI	11
ANALISI DEL RISCHIO	19
ANALISI DEL PROBLEMA	23
Modello del dominio	32
Architettura Logica: Struttura	33
Architettura Logica: Interazione	36
Architettura Logica: Comportamento	40
Piano del lavoro	41
Piano del collaudo	42
PROGETTAZIONE	43
PROGETTAZIONE ARCHITETTURALE	43
Requisiti non funzionali	43
Scelta dell'architettura	44
Scelte tecnologiche	46
PROGETTAZIONE DI DETTAGLIO	47
STRUTTURA	47
INTERAZIONE	60
PROGETTAZIONE DELLA PERSISTENZA	70
PROGETTAZIONE DEL COLLAUDO	72
DEPLOYMENT	75
Artefatti	75
Deployment Type-Level	75

ABSTRACT

Il progetto intende realizzare un'applicazione per la simulazione di un campionato fantabasket.

Il gioco consiste nella competizione di un numero pari di squadre, ognuna gestita da un utente, che sia composta da un numero prestabilito di giocatori reali di una lega cestistica.

Prima dell'inizio del campionato gli utenti formano le squadre con un'asta non gestita dall'applicazione, la quale permetterà però di inserire le squadre già composte.

Ad ogni giornata del campionato reale ne corrisponde una del campionato virtuale, dove ogni utente affronta in scontro diretto un altro utente. Vince colui che ha totalizzato più punti in base alle prestazioni dei giocatori e guadagna dei punti che aggiornano la classifica generale.

Sarà quindi possibile per ogni utente inserire la formazione ad ogni turno scegliendo i giocatori dalla propria rosa.

ANALISI DEI REQUISITI

Raccolta dei requisiti

- L'applicazione deve permettere la simulazione di un campionato di fantabasket tra un numero pari di squadre, ognuna gestita da un utente.
- L'utente accede al sistema con una coppia di credenziali chiamate username e password. Lo username è scelto dall'utente, la password deve essere almeno di 8 caratteri.
- Della creazione del campionato si incarica un utente del gruppo, che ha il ruolo di amministratore della lega, che inserisce tutti gli utenti che fanno parte della sua lega e per ognuno di loro ne inserisce i giocatori della squadra.
- Il numero minimo di squadre per creare un campionato è 2.
- Il committente specifica che l'applicazione gestisca il campionato LBA (Lega Basket A).
- Ad ogni giornata della stagione regolare di LBA ne corrisponde una del campionato virtuale. La fase dei playoff non viene considerata nell'applicazione.
- Le squadre degli utenti sono composte da giocatori reali del campionato LBA, divisi in 5 guardie, 5 ali e 3 centri.
- Un giocatore reale può appartenere al più ad una squadra virtuale all'interno di una lega.
- Nel caso un giocatore non faccia parte di nessuna squadra della lega viene mantenuto in una lista degli svincolati.
- Dopo l'inserimento delle squadre viene generato il calendario casualmente, con il numero di giornate del campionato virtuale che corrisponde al numero effettivo di giornate della LBA ancora da disputare.
- Agli utenti deve essere data la possibilità di iniziare il campionato virtuale a campionato LBA in corso, in questo caso i turni passati non vengono tenuti in conto.
- Il calendario degli scontri diretti viene generato in modo che gli utenti si sfidino esaurendo tutte le combinazioni possibili. Nel caso in cui il campionato LBA non sia terminato si ripetono gli scontri nello stesso ordine.
- Per ogni campionato virtuale viene mantenuta una classifica, in cui per ogni utente i punti classifica sono dati dalla somma di quelli totalizzati nelle varie giornate.
- Per ogni scontro diretto viene calcolato il punteggio delle due squadre e vengono assegnati due punti classifica in caso di vittoria e zero in caso di sconfitta.
Per determinare la vittoria si fa il confronto tra i punteggi delle due squadre, in caso (critico) di parità si procede con il confronto tra i fantapunti dei capitani, poi i fantapunti dei due starting five e infine i fantapunti del sesto uomo.
In caso di totale parità di tutti i parametri si assegna la vittoria alla squadra che gioca in casa (per simulare un vantaggio del fattore casa).
- Un utente vince uno scontro diretto quando totalizza un punteggio più elevato del suo avversario.
- In ogni scontro diretto i fantapunti di un giocatore sono calcolati in base alle sue statistiche nella partita del turno come somma algebrica dei punti segnati, rimbalzi, assist, palle rubate, stoppate, tiri sbagliati, palle perse, falli e una serie di bonus e malus.
- Il peso di ogni statistica può essere scelto dall'amministratore di lega, se non viene specificato quelli di default da applicare sono:
 - Punti: +1
 - Rimbalzo difensivo: +1
 - Rimbalzo offensivo: +1,25
 - Assist: +1,5
 - Palla recuperata: +1,5
 - Palla persa: -1,5

- Stoppata: +1,5
- Doppia doppia: +5
- Tripla doppia: +10
- Quadrupla doppia: +50
- Quintetto base: +1
- Bonus triple segnate 3/4/5+: +3/+4/+5
- Vittoria squadra: +5% del punteggio del giocatore
- Tiro sbagliato: -1
- Tiro libero sbagliato: -1
- Uscita per falli (5): -5.
- Per ogni giornata ogni utente deve inserire la formazione scegliendo il modulo tra i prestabiliti, e inserendo 5 giocatori titolari e 5 dalla panchina, specificando il capitano tra i titolari e il sesto uomo (che fa parte della panchina).
- I titolari e il sesto uomo hanno i fantapunti moltiplicati per 1, il capitano per 2 mentre la panchina per 0,5.
- I moduli prestabiliti sono: 2-2-1, 1-2-2, 2-1-2, 1-3-1, 3-1-1, che rappresentano rispettivamente il numero di guardie, ali e centri da schierare.
- Per ogni giocatore si deve vedere a tempo di inserimento della formazione contro chi giocherà la prossima partita nel campionato LBA.
- L'applicazione non deve permettere l'inserimento della formazione quando manca meno di un'ora all'inizio della prima partita della giornata.
- Nel caso non venga inserita la formazione, viene mantenuta quella della giornata scorsa, e nel caso critico in cui non venga inserita la formazione alla prima giornata ne viene generata una casuale. In entrambi i casi l'amministratore di lega può scegliere se applicare un malus al punteggio dell'utente in quello scontro diretto, che di default è 0.
- Deve essere possibile per gli utenti visualizzare la classifica generale, i risultati degli scontri dei turni precedenti di tutta la sua lega e le rose degli altri utenti in qualsiasi momento, oltre al calendario delle giornate successive.
- Ogni utente deve poter vedere le statistiche medie di ogni giocatore dall'inizio del campionato, sia quelle relative al campionato LBA che quelle in fantapunti.
- Alla fine di ogni turno l'amministratore di lega può dare il comando all'applicazione di calcolare i risultati degli scontri diretti in base alle statistiche dei giocatori.
- L'utente deve poter visualizzare, per ogni giocatore della sua rosa, se e quali sono indisponibili a causa di infortuni o squalifiche.
- L'applicazione si interfaccia con un sistema esterno che per ogni partita del campionato LBA fornisce tutte le statistiche dei giocatori utili a calcolarne il fantapunteggio.
- L'applicazione sarà distribuita e di natura client-server con la presenza di un database per la memorizzazione persistente dei dati.

In un secondo colloquio con il committente sono sorti nuovi requisiti:

- Si vuole dare la possibilità, esclusivamente all'amministratore di lega, di modificare le squadre del campionato per permettere agli utenti di scambiarsi dei giocatori o sostituirli con degli svincolati.
- L'amministratore in qualsiasi momento può modificare il peso da applicare alle varie statistiche nel calcolo dei fantapunti e modificare il malus da applicare al punteggio di un utente in caso di mancato inserimento della formazione entro la scadenza.

Tabella dei Requisiti

Id Requisito	Requisito	Tipo
R1F	Creazione campionato di fantabasket da parte dell'amministratore di lega.	Funzionale
R2F	In ogni campionato il numero minimo di squadre deve essere 2 e sempre un numero pari.	Funzionale
R3F	Gli utenti accedono all'applicazione attraverso una coppia di credenziali username e password.	Funzionale
R4F	Al primo accesso l'utente si deve registrare scegliendo il proprio username e la propria password.	Funzionale
R5F	L'amministrazione di lega inserisce le squadre degli utenti.	Funzionale
R6F	Si possono utilizzare soltanto giocatori del campionato LBA.	Funzionale
R7F	Ad ogni giornata della stagione regolare di LBA ne corrisponde una del campionato virtuale.	Funzionale
R8F	Le squadre degli utenti sono composte da giocatori reali del campionato LBA, divisi in 5 guardie, 5 ali e 3 centri.	Funzionale
R9F	Un giocatore reale può appartenere al più ad una squadra virtuale all'interno di una lega.	Funzionale
R10F	Il calendario degli scontri diretti viene generato casualmente in modo che tutti gli utenti si sfidino esaurendo tutte le combinazioni possibili. Nel caso in cui il campionato LBA non sia terminato si ripetono gli scontri nello stesso ordine.	Funzionale
R11F	Il numero di giornate del campionato virtuale corrisponde al numero effettivo di giornate della LBA ancora da disputare.	Funzionale
R12F	Agli utenti deve essere data la possibilità di iniziare il campionato virtuale a campionato LBA in corso, in questo caso i turni passati non vengono tenuti in conto.	Funzionale
R13F	Per ogni campionato virtuale viene mantenuta una classifica.	Funzionale
R14F	Per ogni scontro diretto vengono assegnati due punti classifica al vincitore e zero al perdente mentre nel caso critico di pareggio entrambi ottengono un punto classifica.	Funzionale
R15F	Un utente vince uno scontro diretto quando totalizza un punteggio più elevato del suo avversario.	Funzionale
R16F	I fantapunti di un giocatore sono calcolati in base alle sue statistiche nella partita del turno.	Funzionale
R17F	Deve essere data la possibilità all'amministratore di lega in qualsiasi momento di scegliere il peso di ogni statistica. Se non viene specificata si applica quella di default.	Funzionale
R18F	Ad ogni giornata, ad ogni utente deve essere consentito di inserire la formazione.	Funzionale
R19F	Inserire la formazione comprende la scelta dei giocatori e la scelta del modulo tra quelli prestabiliti.	Funzionale
R20F	Nella formazione i giocatori schierati sono 5 titolari (tra cui il capitano), un sesto uomo e 4 panchinari.	Funzionale

R21F	Nel calcolo del punteggio di un utente i titolari e il sesto uomo hanno i fantapunti moltiplicati per 1, il capitano per 2 mentre la panchina per 0,5.	Funzionale
R22F	Per ogni giocatore della sua squadra, un utente deve poter vedere a tempo di inserimento della formazione contro chi giocherà la prossima partita nel campionato LBA.	Funzionale
R23F	L'applicazione non deve permettere l'inserimento della formazione quando manca meno di un'ora all'inizio della prima partita della giornata.	Funzionale
R24F	Nel caso in cui un utente non inserisca la formazione entro la scadenza deve essere mantenuta quella della giornata precedente. Nel caso critico in cui non venga inserita la formazione alla prima giornata ne viene generata una casuale.	Funzionale
R25F	Nel caso specificato nel requisito R24F, l'amministratore di lega ha la possibilità di applicare un malus al punteggio dell'utente nello scontro diretto di quella giornata.	Funzionale
R26F	Deve essere possibile per gli utenti visualizzare la classifica generale, i risultati degli scontri dei turni precedenti di tutta la sua lega e le rose degli altri utenti in qualsiasi momento, oltre al calendario delle giornate successive.	Funzionale
R27F	Ogni utente deve poter vedere le statistiche medie di ogni giocatore dall'inizio del campionato, sia quelle relative al campionato LBA che quelle in fantapunti.	Funzionale
R28F	Alla fine di ogni turno l'amministratore di lega può dare il comando all'applicazione di calcolare i risultati degli scontri diretti in base alle statistiche dei giocatori.	Funzionale
R29F	L'utente deve poter visualizzare, per ogni giocatore della sua rosa, se e quali sono indisponibili a causa di infortuni o squalifiche.	Funzionale
R30F	L'amministratore di lega può modificare le squadre del campionato	Funzionale
R31F	Il malus da applicare nel caso visto al requisito R25F può essere impostato in qualsiasi momento.	Funzionale
R32F	Deve essere mantenuta una lista dei giocatori svincolati del campionato	Funzionale
R33F	Un utente può partecipare al massimo ad una sola lega	Funzionale
R34F/R1NF	L'applicazione si interfaccia con un sistema esterno, che non compare in questa analisi, e che per ogni partita del campionato LBA fornisce tutte le statistiche dei giocatori utili a calcolarne i fantapunti, gli orari delle partite del turno successivo, gli eventuali infortunati e squalificati, e gli eventuali trasferimenti che si sono stati dall'ultima giornata giocata	Funzionale/Non Funzionale
R35F/R2NF	L'applicazione si interfaccia di un sistema esterno, che non compare in questa analisi, e che all'inizio del campionato LBA fornisce il calendario delle partite e la lista dei giocatori di tutte le squadre	Funzionale/Non Funzionale
R3NF	L'applicazione distribuita di natura client server con la presenza di un database per la memorizzazione persistente dei dati.	Non Funzionale

R4NF	Velocità di ricerca dei dati.	Non Funzionale
R5NF	L'applicazione deve essere intuitiva perché è destinata ad utenti non tecnici.	Non Funzionale
R6NF	Protezione dei dati	Non Funzionale
R7NF	Per sapere le date e gli orari delle partite l'applicazione si serve di un sistema esterno	Non Funzionale
R8NF	Non deve succedere che un utente possa svolgere operazioni per conto di altri utenti, comprese quelle di amministratore	Non Funzionale

ANALISI DEL DOMINIO

Vocabolario

Voce	Definizione	Sinonimi
punti	Punti segnati dai giocatori reali nella partita del turno	
fantapunti	Somma algebrica delle statistiche moltiplicate per i coefficienti, con l'aggiunta di bonus e malus che un giocatore reale totalizza in una giornata di campionato LBA	
punteggio	Somma dei fantapunti dei giocatori schierati da un utente in una partita di campionato. I fantapunti dei titolari e del sesto uomo vengono moltiplicati per 1, quelli del capitano per 2 e quelli dei panchinari per 0,5	
punti classifica	Vengono assegnati ad un utente al termine di uno scontro diretto, 2 in caso di vittoria e 0 per la sconfitta	
campionato virtuale	Campionato di fantabasket gestito dall'applicazione	campionato, lega
campionato LBA	Campionato di Lega Basket serie A	
utente	Utilizzatore dell'applicazione, colui che gestisce una squadra del campionato virtuale	
giocatore	Colui che gioca in una squadra reale del campionato LBA	
amministratore di lega	Utente che crea il campionato e inserisce le squadre degli altri utenti nell'applicazione. Lui stesso gestisce una squadra della sua lega.	
squadra	Insieme di giocatori gestiti da un utente	rosa
calendario	Insieme delle giornate, dall'inizio alla fine del campionato virtuale	
giornata	ciascuno dei giorni e le relative serie di scontri diretti del campionato virtuale fissati nel calendario	turno
partita	Confronto tra due squadre reali del campionato LBA	
scontro diretto	Confronto tra le squadre di due utenti del campionato	
classifica	Elenco ordinato degli utenti di una lega in base ai rispettivi punti classifica	
formazione	Insieme di giocatori che un utente schiera per il turno attuale, scegliendoli dalla propria squadra. In particolare, ogni formazione è composta da 5 titolari, un sesto uomo e 4 panchinari. Tra i titolari deve essere specificato il capitano	
svincolato	Giocatore reale che non appartiene a nessuna squadra della lega	
modulo	Combinazione del numero di guardie, ali e centri	

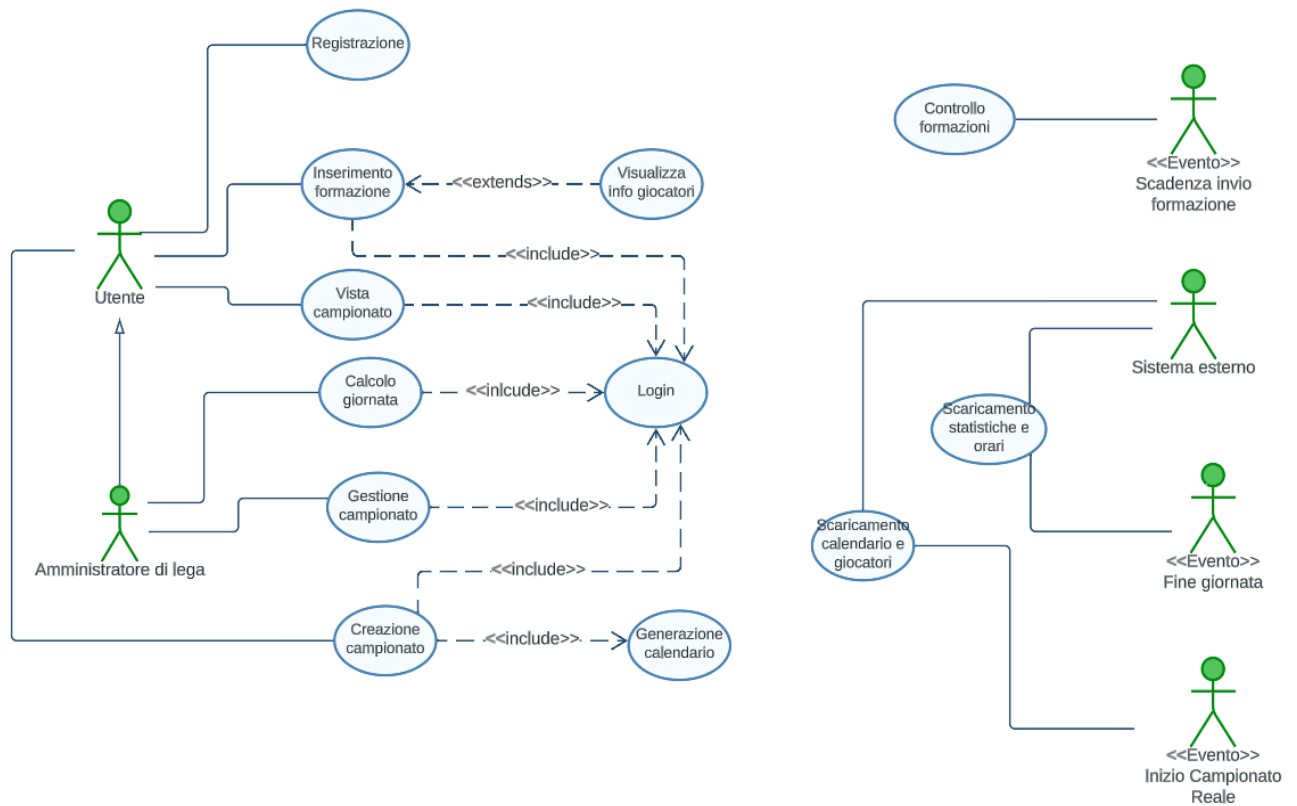
	schierati nei titolari della formazione	
risultato	Confronto tra i punteggi di due utenti che si affrontano in scontro diretto	
credenziali	Insieme composto da username e password necessari per accedere al sistema	
username	È una stringa alfanumerica scelta dall'utente di massimo 64 caratteri. Essa è univoca all'interno dell'applicazione.	
password	Codice alfanumerico di almeno 8 caratteri	

Sistemi esterni

La nostra applicazione si interfaccia con un sistema esterno che fornisce le statistiche di tutti i giocatori per ogni partita del campionato LBA. Inoltre, all'inizio della stagione LBA fornisce il calendario del campionato e la lista dei giocatori di tutte le squadre. Ad ogni giornata è anche in grado comunicare i trasferimenti avvenuti dalla giornata precedente e gli orari delle partite del turno successivo.

ANALISI DEI REQUISITI

Casi d'uso



Il diagramma dei casi d'uso mostra come la nostra applicazione sia interessata a tre diversi eventi: l'evento di scadenza invio formazione, che segnala quando manca un'ora all'inizio della prima partita della giornata, l'evento di fine giornata e l'evento di inizio campionato reale. In particolare, in questi ultimi due eventi viene coinvolto il sistema esterno per usufruire delle sue funzionalità.

Scenari

Titolo	Login
Descrizione	Permette di accedere al sistema
Attori	Utente
Relazioni	Inserimento formazione, Vista campionato, Calcolo giornata, Gestione campionato, Creazione campionato
Precondizioni	L'utente ha effettuato la registrazione
Postcondizioni	
Scenario principale	1. L'utente inserisce le proprie credenziali di accesso al sistema 2. Il sistema dopo le opportune verifiche sulla correttezza e la validità delle credenziali, permette all'utente di accedere all'applicazione
Scenari alternativi	Scenario A: credenziali non riconosciute 2. Il sistema dopo le opportune verifiche non riconosce i dati immessi e presenta nuovamente la schermata iniziale di accesso
Requisiti non funzionali	Protezione dei dati
Punti aperti	

Titolo	Registrazione
Descrizione	Permette di registrare le proprie credenziali nel sistema
Attori	Utente
Relazioni	
Precondizioni	
Postcondizioni	Le credenziali inserite vengono salvate dal sistema
Scenario principale	1. L'utente sceglie uno username ed una password. 2. Il sistema controlla che lo username sia di massimo 64 caratteri alfanumerici e che non sia già utilizzato nel sistema. Inoltre, controlla anche che la password sia di minimo 8 caratteri alfanumerici. 3. Il sistema dopo le opportune verifiche sulla correttezza delle credenziali inserite, presenta la pagina iniziale dell'applicazione.
Scenari alternativi	Scenario A: username già esistente 3. Il sistema comunica all'utente che lo username è già in uso e di sceglierne uno diverso. Scenario B: username o password non rispettano i requisiti del sistema 3. Il sistema comunica all'utente che lo username e/o la password non soddisfano i requisiti e di sceglierne di diversi.
Requisiti non funzionali	Protezione dei dati
Punti aperti	

Titolo	Inserimento formazione
Descrizione	Permette ad un utente di inserire la propria formazione per lo scontro diretto della prossima giornata.
Attori	Utente
Relazioni	Visualizza info giocatori, Login
Precondizioni	1. L'utente fa parte di un campionato 2. Manca almeno un'ora all'inizio della prima partita della giornata
Postcondizioni	La formazione inserita viene salvata dal sistema
Scenario principale	1. Login 2. Il sistema verifica se l'utente ha precedentemente inserito una formazione per il prossimo turno, e nel caso la mostra 3. L'utente inserisce la formazione potendo controllare per ogni giocatore della sua rosa se è indisponibile e contro chi giocherà la prossima partita nel campionato LBA 4. L'utente salva la formazione inserita
Scenari alternativi	
Requisiti non funzionali	Intuitività nell'uso delle funzionalità
Punti aperti	

Titolo	Visualizza info giocatori
Descrizione	Permette di visualizzare per ogni giocatore della propria rosa la disponibilità e l'avversario che dovrà affrontare nella prossima partita di campionato LBA
Attori	Utente
Relazioni	Inserimento formazione
Precondizioni	
Postcondizioni	
Scenario principale	1. L'utente seleziona un giocatore della propria rosa di cui vuole visualizzare le informazioni 2. L'applicazione mostra all'utente le informazioni per il giocatore richiesto
Scenari alternativi	
Requisiti non funzionali	Intuitività nell'uso delle funzionalità
Punti aperti	

Titolo	Vista campionato
Descrizione	Permette all'utente di visualizzare la classifica del campionato, i risultati degli scontri diretti delle giornate passate, il calendario del campionato e le statistiche dei giocatori dall'inizio del campionato
Attori	Utente
Relazioni	Login
Precondizioni	1. L'utente fa parte di un campionato 2. Le statistiche dei giocatori sono state scaricate 3. Il calendario è stato generato
Postcondizioni	
Scenario principale	1. Login 2. All'utente viene presentato un menù, dove è possibile scegliere che cosa visualizzare tra classifica del campionato, statistiche dei giocatori, calendario, risultati e rose degli altri utenti 3. L'utente sceglie di visualizzare la classifica 4. L'applicazione mostra all'utente ciò che ha richiesto
Scenari alternativi	Scenario A: 3. L'utente sceglie di visualizzare le statistiche dei giocatori, si continua dal punto 4 dello scenario principale Scenario B: 3. L'utente sceglie di visualizzare il calendario e i risultati, si continua dal punto 4 dello scenario principale Scenario C: 3. L'utente sceglie di visualizzare le rose degli altri utenti della lega
Requisiti non funzionali	Intuitività nell'uso delle funzionalità
Punti aperti	

Titolo	Calcola giornata
Descrizione	Permette all'amministratore di lega di ottenere i risultati degli scontri diretti della giornata corrente
Attori	Amministratore di lega
Relazioni	Login
Precondizioni	Che sia terminata l'ultima partita della giornata
Postcondizioni	I risultati degli scontri diretti sono disponibili nell'applicazione
Scenario principale	1. Login 2. L'amministratore di lega richiede il calcolo dei risultati 3. L'applicazione usa le statistiche scaricate precedentemente per calcolare i fantapunti dei giocatori 4. In base ai fantapunti dei giocatori schierati viene calcolato il punteggio di ogni utente del campionato 5. Viene applicato il malus nel caso in cui l'utente non ha schierato la formazione

	6. Vengono assegnati i punti classifica per ogni scontro diretto 7. Viene aggiornata la classifica 8. Viene salvato tutto in modo persistente
Scenari alternativi	
Requisiti non funzionali	Velocità di ricerca dei dati
Punti aperti	

Titolo	Gestione Campionato
Descrizione	Questa funzionalità permette all'amministratore di lega di modificare le squadre della lega, modificare il peso da applicare alle varie statistiche nel calcolo dei fantapunti e modificare il malus da applicare al punteggio di un utente in caso di mancato inserimento della formazione entro la scadenza
Attori	Amministratore di lega
Relazioni	Login
Precondizioni	L'amministratore ha generato il campionato e inserito le squadre degli utenti
Postcondizioni	
Scenario principale	1. Login 2. Viene presentato un menù, dove è possibile scegliere tra modifica di una squadra, modifica del peso delle statistiche e modifica del malus 3. L'amministratore sceglie di modificare una squadra 4. L'amministratore sceglie quale squadra modificare 5. L'amministratore rimuove dalla rosa un giocatore 6. L'amministratore può scegliere se ripartire dal punto 4 oppure inserire un giocatore dalla lista svincolati 7. L'amministratore salva la modifica
Scenari alternativi	Scenario A: modifica del peso delle statistiche 3. L'amministratore sceglie di modificare il peso delle statistiche 4. L'amministratore sceglie quale peso modificare 5. Modifica il peso 6. può ripartire dal punto 4 oppure salvare le modifiche Scenario B: modifica malus 3. L'amministratore sceglie di modificare il malus 4. L'amministratore modifica il malus 5. L'amministratore salva la modifica
Requisiti non funzionali	Intuitività nell'uso delle funzionalità
Punti aperti	

Titolo	Creazione Campionato
Descrizione	Permette ad un utente che non fa parte di nessuna lega di crearne una nuova e di inserire tutti gli utenti con le rispettive squadre
Attori	Utente
Relazioni	Login, Generazione calendario
Precondizioni	
Postcondizioni	
Scenario principale	1. L'utente può inserire i pesi delle statistiche nel calcolo dei fantapunti di un giocatore 2. L'utente può inserire il malus relativo al mancato schieramento della formazione 3. Inserisce il numero di utenti che comporranno la lega 4. Inserisce la lista degli utenti identificati per username 5. Per ogni utente, inserisce i giocatori della sua squadra rispettando i vincoli sui ruoli 6. Generazione calendario 7. Salva modifiche
Scenari alternativi	
Requisiti non funzionali	Intuitività nell'uso delle funzionalità
Punti aperti	

Titolo	Generazione calendario
Descrizione	Generazione automatica del calendario della lega
Attori	Amministratore di lega
Relazioni	Creazione campionato
Precondizioni	
Postcondizioni	
Scenario principale	1. Il sistema genera automaticamente tutti gli scontri diretti di tutte le giornate del campionato 2. Il calendario viene salvato in modo persistente
Scenari alternativi	
Requisiti non funzionali	
Punti aperti	

Titolo	Scaricamento calendario e giocatori
Descrizione	L'applicazione si serve di un sistema esterno per ottenere il calendario del campionato LBA e la lista dei giocatori di tutte le squadre
Attori	Sistema esterno, evento inizio campionato reale
Relazioni	
Precondizioni	
Postcondizioni	
Scenario principale	1. L'applicazione richiede al sistema esterno lo scaricamento dei dati 2. Il sistema esterno fornisce i dati richiesti
Scenari alternativi	
Requisiti non funzionali	
Punti aperti	

Titolo	Scaricamento statistiche e orari
Descrizione	L'applicazione si serve di un sistema esterno per ottenere le statistiche dei giocatori totalizzate nell'ultima giornata di campionato LBA, oltre agli orari delle partite della giornata successiva del campionato LBA. Inoltre si aggiorna la lista dei giocatori nel caso in cui ci siano stati dei trasferimenti dall'ultima giornata giocata
Attori	Sistema esterno, evento fine giornata
Relazioni	
Precondizioni	
Postcondizioni	
Scenario principale	1. L'applicazione richiede al sistema esterno lo scaricamento dei dati 2. Il sistema esterno fornisce i dati richiesti
Scenari alternativi	
Requisiti non funzionali	
Punti aperti	

Titolo	Controllo formazione
Descrizione	L'applicazione controlla automaticamente che tutti gli utenti di tutte le leghe abbiano inserito la formazione, e in caso negativo provvede ad inserirne una
Attori	evento fine scadenza invio formazione
Relazioni	
Precondizioni	
Postcondizioni	
Scenario principale	Per ogni utente: 1. L'applicazione controlla che sia stata inserita la formazione per la giornata attuale 2. La validazione va a buon fine
Scenari alternativi	Scenario A: l'utente non ha inserito la formazione e non è alla prima giornata 2. viene inserita la formazione della giornata precedente 3. viene salvata l'informazione relativa al mancato schieramento Scenario B: l'utente non ha inserito la formazione ed è alla prima giornata 2. viene generata una formazione casuale 3. viene salvata l'informazione relativa al mancato schieramento
Requisiti non funzionali	
Punti aperti	

ANALISI DEL RISCHIO

Tabella Valutazione dei Beni

Bene	Valore	Esposizione
Sistema informativo	Alto, supporto all'intera applicazione	Alta, perdita di immagine e finanziaria. Impossibilità di garantire la prosecuzione delle leghe iniziate
Credenziali utenti	Alto, permettono agli utenti di accedere al sistema, e nel caso di un amministratore anche di apportare modifiche alla lega	Alta, perdita d'immagine e accesso degli utenti compromesso
Informazioni sulla lega	Alto, mantengono la classifica, i risultati e il calendario generato di una lega	Alta, perdita d'immagine ed impossibilità di garantire la prosecuzione delle leghe iniziate
Informazioni sulle squadre	Medio, elenco dei giocatori appartenenti alle varie squadre delle leghe	Media, l'alterazione delle squadre comporta l'intervento manuale dell'amministratore di lega
Statistiche giocatori	Basso, rappresentano soltanto una copia delle informazioni che sono mantenute dal sistema esterno	Media, la loro alterazione comporta di richiederle nuovamente al sistema esterno
Impostazioni lega	Basso, contengono i pesi scelti per il calcolo dei fantapunti e il malus da applicare in caso di mancato schieramento delle formazioni	Medio, richiedono all'amministratore di lega di impostarle nuovamente

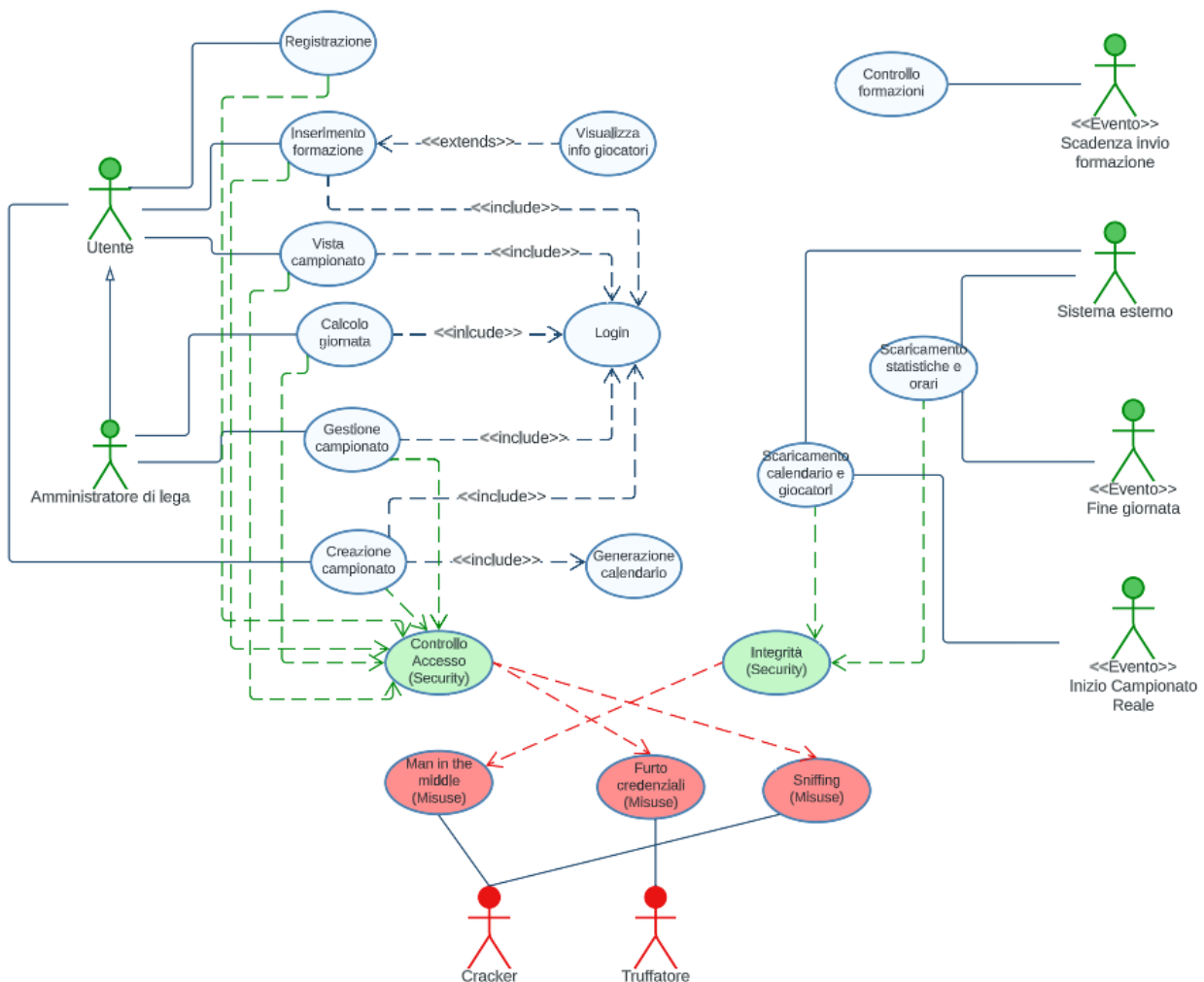
Tabella Minacce/Controlli

Minaccia	Probabilità	Controllo	Fattibilità
Furto credenziali utente	Media	Log delle operazioni	Basso costo implementativo
Furto credenziali amministratore di lega	Media	Log delle operazioni	Basso costo implementativo
Intercettazione comunicazioni	Media, il sistema è distribuito e avvengono numerose interazioni tra clienti e servitore	Cifratura delle comunicazioni	Il costo dipende dal tipo di cifratura scelto. Se simmetrica basso, se asimmetrica più alto dovuto alla necessità di rilascio di coppie di chiavi da un ente di certificazione
		Log di tutte le operazioni	Basso costo implementativo
DoS	Bassa	Progettazione opportuna, controllo e limitazione degli accessi	Costo basso. Impossibile prevenire questo tipo di attacco
Man in the middle	Bassa	Autenticazione del sistema esterno che fornisce le statistiche tramite certificati	Costo basso

Analisi Tecnologica della Sicurezza

Tecnologia	Vulnerabilità
Autenticazione tramite username e password	- L'utente sceglie una password banale - Utente rivela volontariamente la password - Utente rivela la password con un attacco di ingegneria sociale
Cifratura comunicazioni	Le vulnerabilità dipendono dal tipo di cifratura. Cifratura Simmetrica: - Tempo di vita della chiave. Più informazioni vengono cifrate con la stessa chiave, più materiale è offerto per l'analisi del testo a un attaccante - Lunghezza della chiave - Memorizzazione della chiave Cifratura Asimmetrica: - Memorizzazione chiave privata
Architettura Client-Server	- DoS - Man in the Middle - Sniffing delle comunicazioni

Security Use Case & Misuse Case



Security Use Case & Misuse Case - Scenari

Titolo	Controllo accesso	
Descrizione	Controllo di ogni accesso al sistema	
Misuse case	Furto di credenziali, sniffing	
Relazioni		
Precondizioni	L'attaccante ha i mezzi per ottenere le credenziali di un utente o di un amministratore di lega	
Postcondizioni	Il sistema blocca momentaneamente l'accesso all'utente o all'amministratore e notifica un tentativo di accesso fraudolento	
Scenario principale	<i>Sistema</i>	<i>Attaccante</i>
		Conoscendo uno username, l'attaccante tenta di indovinare la password per accedere al sistema con un attacco di forza bruta
	Il sistema controlla le credenziali inserite e scrive nel log il tentato accesso. Al quinto tentativo di accesso errato blocca temporaneamente l'utente.	
Scenario di attacco avvenuto con successo	<i>Sistema</i>	<i>Attaccante</i>
		Effettua il login
	Il sistema controlla le credenziali inserite e consente l'accesso al sistema	
		L'attaccante opera nel sistema con l'identità dell'utente con cui si è autenticato
	Il sistema scrive nel log tutte le operazioni eseguite dall'attaccante	

Titolo	Integrità	
Descrizione	Integrità dei dati trasmessi dal Sistema esterno	
Misuse case	Man in the middle	
Relazioni		
Precondizioni	1. L'attaccante ha la possibilità di intercettare i dati inviati dal sistema esterno all'applicazione 2. L'attaccante ha la possibilità di modificare i dati intercettati 3. L'attaccante ha la possibilità di ritrasmettere i dati modificati all'applicazione	
Postcondizioni		
Scenario principale	<i>Sistema</i>	<i>Attaccante</i>
		L'attaccante intercetta e modifica i dati inviati dal sistema esterno
	Il Sistema rileva che i dati sono stati corrotti e registra il tentativo di attacco nel log	
Scenario di attacco avvenuto con successo	<i>Sistema</i>	<i>Attaccante</i>
		L'attaccante intercetta e modifica i dati inviati dal sistema esterno
	Il Sistema elabora i dati ricevuti non rilevando la loro incorrettezza. Il sistema registra l'operazione nel log	

Requisiti di Protezione dei Dati

Dall'analisi del rischio sono emersi i seguenti ulteriori requisiti:

1. Creazione di un sistema di log per tracciare tutte le azioni che avvengono sul sistema. Si introduce un nuovo attore, admin di sistema, che ha come funzionalità la lettura dei log. Questo implica l'aggiunta di un requisito (R36F) per la lettura, e un altro per la scrittura (R37F)
2. Individuare una corretta politica di controllo degli accessi
3. I dati memorizzati nel sistema devono essere protetti
4. I dati trasmessi dal sistema esterno devono essere protetti da attacchi 'Man in the middle', adottando meccanismi di cifratura e autenticazione.

ANALISI DEL PROBLEMA

Analisi Documento dei Requisiti: Analisi delle Funzionalità

Tabella Funzionalità

Funzionalità	Tipo	Grado Complessità	Requisiti collegati
Registrazione	Memorizzazione dei dati	Semplice	R4F
InserimentoFormazione	Interazione con l'esterno, gestione e memorizzazione dei dati	Complessa	R18F, R19F, R20F, R22F, R23F, R29F
VistaCampionato	Interazione con l'esterno	Semplice	R13F, R26F, R27F
CalcoloGiornata	Manipolazione e memorizzazione dati	Complessa	R14F, R15F, R16F, R21F, R28F
Login	Interazione con l'esterno, gestione dati	Semplice	R3F
GestioneCampionato	Gestione dati	Complessa	R17F, R25F, R30F, R31F, R32F
CreazioneCampionato	Memorizzazione dati	Complessa	R1F, R2F, R5F-R12F, R17F, R25F
ScaricamentoStatisticheOrari	Interazione con l'esterno, memorizzazione dati	Semplice	R6F, R16F, R34F
ScaricamentoCalendarioGiocatori	Interazione con l'esterno, memorizzazione dati	Semplice	R35F
ControlloFormazioni	Gestione dati	Semplice	R24F
ScritturaLog	Memorizzazione dati	Semplice	R37F
VisualizzaLog	Gestione dati	Semplice	R36F

Registrazione: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Username	Semplice	Protezione molto alta	Input	Massimo 64 caratteri alfanumerici
Password	Semplice	Protezione molto alta	Input	Minimo 8 caratteri alfanumerici, massimo 64

InserimentoFormazione: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Modulo	Semplice	Protezione bassa	Input	
Titolari	Composto	Protezione bassa	Input	Esattamente 5 giocatori
Panchina	Composto	Protezione bassa	Input	Esattamente 5 giocatori
Capitano	Semplice	Protezione bassa	input	Il giocatore deve essere titolare
Sesto uomo	Semplice	Protezione bassa	Input	Il giocatore deve essere in panchina

VistaCampionato: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Classifica	Composto	Protezione bassa	Output	
Calendario	Composto	Protezione bassa	Output	
Scontri Diretti	Composto	Protezione bassa	Output	
Statistiche	Composto	Protezione bassa	Output	
Rose utenti	Composto	Protezione basse	Output	

CalcoloGiornata: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Scontri Diretti	Composto	Protezione bassa	Input	
Formazioni	Composto	Protezione bassa	Input	
Punteggi	Composto	Protezione bassa	Output	
Fantapunti giocatori	Composto	Protezione bassa	Output	
Statistiche della giornata	Composto	Protezione bassa	Input	
Pesi statistiche	Composto	Protezione bassa	Input	
Malus	Semplice	Protezione bassa	Input	
Utenti che non hanno inserito la formazione	Composto	Protezione bassa	Input	

Login: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Username	Semplice	Protezione molto alta	Input	Massimo 64 caratteri alfanumerici
Password	Semplice	Protezione molto alta	Input	Minimo 8 caratteri alfanumerici, massimo 64

GestioneCampionato: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Squadra da modificare	Composto	Protezione bassa	Input	
Giocatore da rimuovere	Semplice	Protezione bassa	Input	Deve far parte della squadra scelta
Giocatore da aggiungere	Semplice	Protezione bassa	Input	Deve essere nella lista svincolati
Malus	Semplice	Protezione bassa	Input	
Peso statistiche da modificare	Composto	Protezione bassa	Input	

CreazioneCampionato: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Utenti	Composto	Protezione bassa	Input	
Squadre	Composto	Protezione bassa	Input	
Pesi statistiche	Composto	Protezione bassa	Input	
Malus	Semplice	Protezione bassa	Input	
Calendario	Composto	Protezione bassa	Output	

ScaricamentoStatisticheOrari: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Statistiche	Composto	Protezione alta	Output	
Orari	Composto	Protezione alta	Output	
Trasferimenti	Composto	Protezione alta	Output	

ScaricamentoCalendarioGiocatori: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Calendario	Composto	Protezione alta	Output	
Giocatori	Composto	Protezione alta	Output	

ControlloFormazioni: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Formazione inserita	Semplice	Protezione bassa	Input	Valore booleano
Esito	Semplice	Protezione bassa	Output	

ScritturaLog: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Data	Semplice	Protezione media	Input	Non più di 20 caratteri
Ora	Semplice	Protezione media	Input	Non più di 20 caratteri
Operazione eseguita	Composto	Protezione alta	Input	

VisualizzaLog: Tabella Informazioni/Flusso

Informazione	Tipo	Livello protezione / privacy	Input / Output	Vincoli
Data	Semplice	Protezione media	Input	
Ora	Semplice	Protezione media	Input	
Operazione eseguita	Composto	Protezione alta	Input	

Analisi Documento dei Requisiti: Analisi dei Vincoli

Tabella Vincoli

Requisito	Categorie	Impatto	Funzionalità
Velocità di ricerca dati (R3NF)	Tempo di risposta	Cercare di migliorare	InserimentoFormazione, VistaCampionato, CalcolaGiornata, GestioneCampionato, Login, ControlloFormazioni
Intuitività d'uso (R4NF)	Usabilità	Migliorare esperienza utente	InserimentoFormazione, VistaCampionato, CalcolaGiornata, GestioneCampionato, Registrazione, Login, CreazioneCampionato
Protezione e integrità dei dati (R1NF,R2NF,R5NF, R6NF)	Sicurezza	Garantire una corretta trasmissione e memorizzazione dei dati, aumentando però i tempi di risposta	InserimentoFormazione, CalcolaGiornata, GestioneCampionato, Registrazione, Login, CreazioneCampionato ScaricamentoStatisticheOrari, ScaricamentoCalendarioGiocatori
Controllo Accessi (R7NF)	Sicurezza	Aumentano tempi di risposta e usabilità ma migliorano la privacy dei dati	InserimentoFormazione, GestioneCampionato, Login, CalcolaGiornata

Analisi Documento dei Requisiti: Analisi delle Interazioni

Tabella Maschere

Maschera	Informazioni	Funzionalità
View Registrazione	Username, password, possibilità di navigare alla view Login	Registrazione
View Login	Username, password, possibilità di navigare alla view Registrazione	Login
View Formazione	Se è già stata inserita una formazione per il prossimo turno la si mostra all'utente, altrimenti si mostra un messaggio che inviti a inserirla. In entrambi i casi viene data la possibilità di spostarsi nella view inserimento formazione	InserimentoFormazione
View InserimentoFormazione	Scelta di un modulo dalla lista dei moduli predefiniti. Scelta dei titolari, della panchina, del capitano e del sesto uomo dalla lista dei giocatori della propria rosa. Per ogni giocatore si può scegliere di visualizzare la view informazioni giocatore	InserimentoFormazione
View InformazioniGiocatore	Disponibilità/indisponibilità di un giocatore, prossima squadra avversaria del campionato LBA	InserimentoFormazione
Home VistaCampionato	Possibilità di navigare verso le view StatisticheGiocatori, Classifica, Calendario, Svincolati e Formazione	VistaCampionato
View StatisticheGiocatori	Statistiche dei giocatori di ogni squadra	VistaCampionato
View Classifica	Classifica della lega e possibilità di poter vedere le squadre di ogni utente attraverso View Squadra	VistaCampionato
View Calendario	Calendario del campionato virtuale con i risultati dei turni già giocati	VistaCampionato
View Squadra	Elenco dei giocatori appartenenti alla squadra di ogni utente	VistaCampionato
Home GestioneCampionato	Possibilità di navigare verso la view ModificaSquadra e PesìStatistiche. Possibilità di calcolare la giornata	GestioneCampionato, CalcolaGiornata
View Svincolati	Lista dei giocatori svincolati nella lega	VistaCampionato
View ModificaSquadra	Squadra da modificare, giocatore da rimuovere, giocatore da aggiungere, lista degli svincolati	GestioneCampionato
View PesìStatistiche	Elenco dei tipi di statistiche con relativi campi per l'inserimento dei pesi. Campo di inserimento del malus.	GestioneCampionato
View CreazioneCampionato	Lista degli utenti della lega, giocatori della squadra di ogni utente, pesi delle statistiche, malus	CreazioneCampionato
View Log	Lista dei log con data, ora e operazione eseguita	VisualizzaLog

Tabella Sistemi Esterni

Sistema	Descrizione	Protocollo di Interazione	Livello di Sicurezza
Recapito dati	Sistema che si occupa di fornire all'applicazione le statistiche di tutti i giocatori per ogni partita del campionato LBA. Inoltre, fornisce le date e gli orari delle partite del campionato LBA e la lista dei giocatori delle squadre	L'applicazione interagisce con il sistema per chiedere le date e orari oppure le statistiche relative ad un turno specifico	Medio-alta, bisogna verificare l'autenticità e integrità delle informazioni ricevute

Analisi Ruoli e Responsabilità

Tabella Ruoli

Ruolo	Responsabilità	Maschere	Riservatezza	Numerosità
Utente	Partecipare alla lega inserendo la formazione ad ogni giornata. Può creare un campionato e diventarne amministratore	View registrazione, View Login, View Formazione, View InserimentoFormazione, Home VistaCampionato, View StatisticheGiocatori, View Classifica, View Calendario, View Svincolati View Squadra, View CreazioneCampionato	Richiesto un livello medio/ basso di riservatezza	Potenzialmente molto alto
Amministratore di lega	Partecipa ad una lega come un qualsiasi utente, inoltre ha le responsabilità di calcolare le giornate e gestire la lega	View registrazione, View Login, View Formazione, View InserimentoFormazione, Home VistaCampionato, View StatisticheGiocatori, View Classifica, View Calendario, View Svincolati View Squadra, Home GestioneCampionato, View ModificaSquadra, View Pesistatistiche	Richiesto un medio livello di riservatezza	Uno per ogni lega, si stima non oltre il migliaio
Admin di sistema	Visualizzazione log	View Log, View Login	Richiesto un alto livello di riservatezza	2/3 persone considerando un orario lavorativo di 8-10 ore al giorno

Utente: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Username	Scrittura
Password	Scrittura
Formazione prossimo turno	Lettura/Scrittura
Statistiche giocatore	Lettura
Disponibilità e prossimo avversario del campionato LBA di un giocatore	Lettura
Classifica	Lettura
Squadre della lega	Lettura
Calendario	Lettura
Lista svincolati	Lettura

Amministratore di lega: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Username	Scrittura
Password	Scrittura
Formazione prossimo turno	Lettura/Scrittura
Statistiche giocatore	Lettura
Disponibilità e prossimo avversario del campionato LBA di un giocatore	Lettura
Classifica	Lettura
Squadre della lega	Lettura/Scrittura
Calendario	Lettura
Lista Svincolati	Lettura/Scrittura
Pesi statistiche	Lettura/Scrittura
Malus	Lettura/Scrittura
Utenti della lega	Scrittura

Admin di sistema: Tabella Ruolo-Informazioni

Informazione	Tipo di Accesso
Log	Lettura
Username	Scrittura
Password	Scrittura

Scomposizione del Problema

Tabella Scomposizione Funzionalità

Funzionalità	Scomposizione
InserimentoFormazione	SceltaModulo InserimentoGiocatori VisualizzaInfoGiocatori
GestioneCampionato	ModificaSquadre ModificaPesoStatistiche ModificaMalus
CreazioneCampionato	InserimentoPesiStatistiche InserimentoMalus InserimentoUtenti InserimentoSquadre GenerazioneCalendario

InserimentoFormazione: Tabella Sotto-Funzionalità

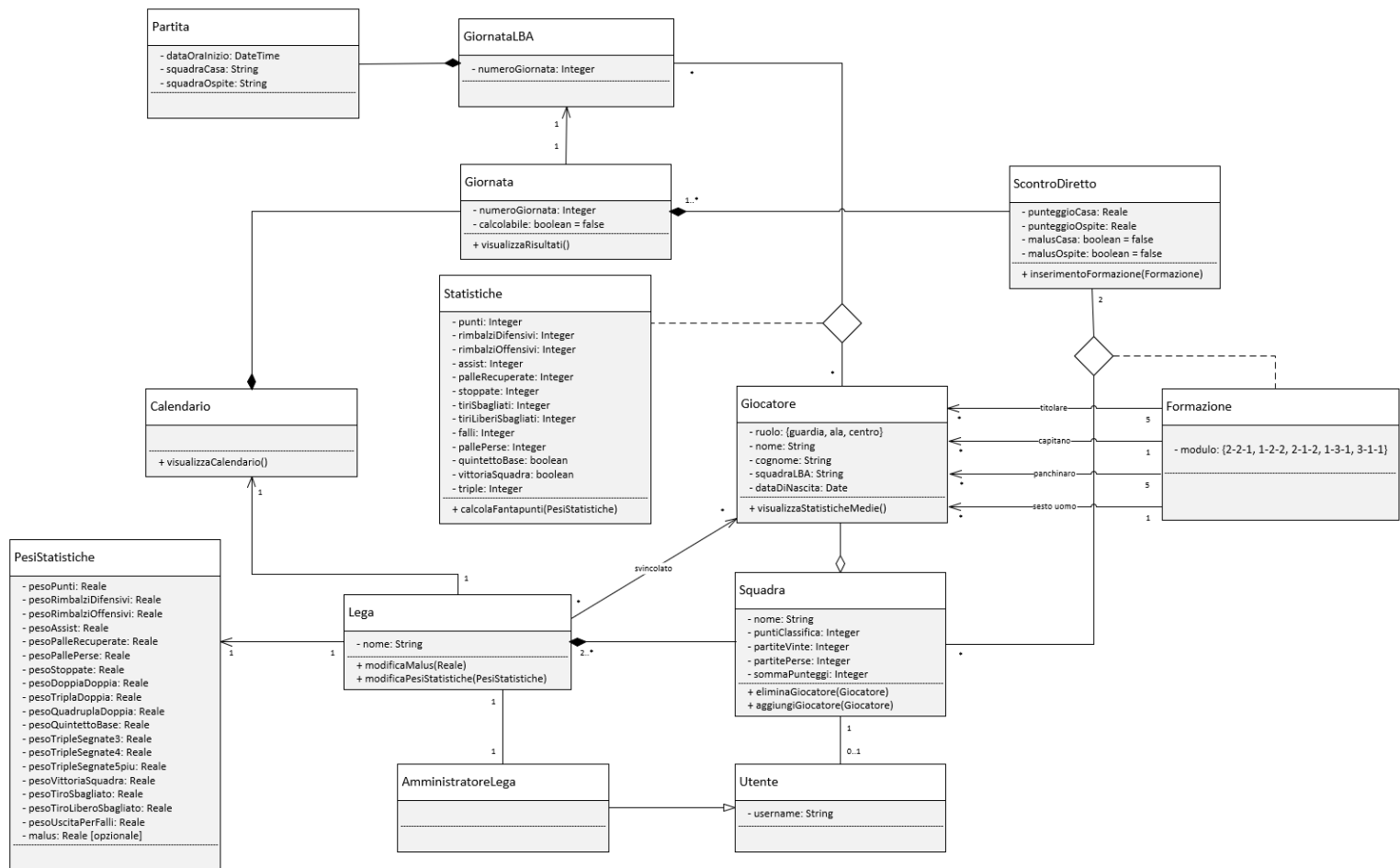
Sotto-funzionalità	Sotto-funzionalità	Legame	Informazioni
SceltaModulo	InserimentoGiocatori	Non è possibile inserire i giocatori se prima non si è scelto il modulo	Modulo
InserimentoGiocatori	VisualizzaInfoGiocatori	Deve essere possibile per l'utente visualizzare le informazioni di un giocatore al momento di inserimento della formazione	Identificativo giocatore

CreazioneCampionato: Tabella Sotto-Funzionalità

Sotto-funzionalità	Sotto-funzionalità	Legame	Informazioni
InserimentoUtenti	InserimentoSquadre	Non è possibile inserire le squadre se prima non sono stati inseriti gli utenti	Utenti
GenerazioneCalendario	InserimentoUtenti	Non è possibile generare il calendario se prima non sono stati inseriti gli utenti	Utenti

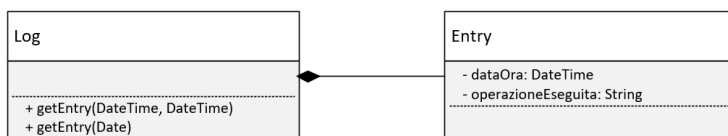
Modello del dominio

Il seguente diagramma delle classi rappresenta il modello del dominio dell'applicazione



Precisazione: le molteplicità delle associazioni sono in notazione BCP

Il seguente diagramma delle classi rappresenta la parte di modello del dominio relativo alla gestione dei log



Architettura Logica: Struttura Diagramma dei package

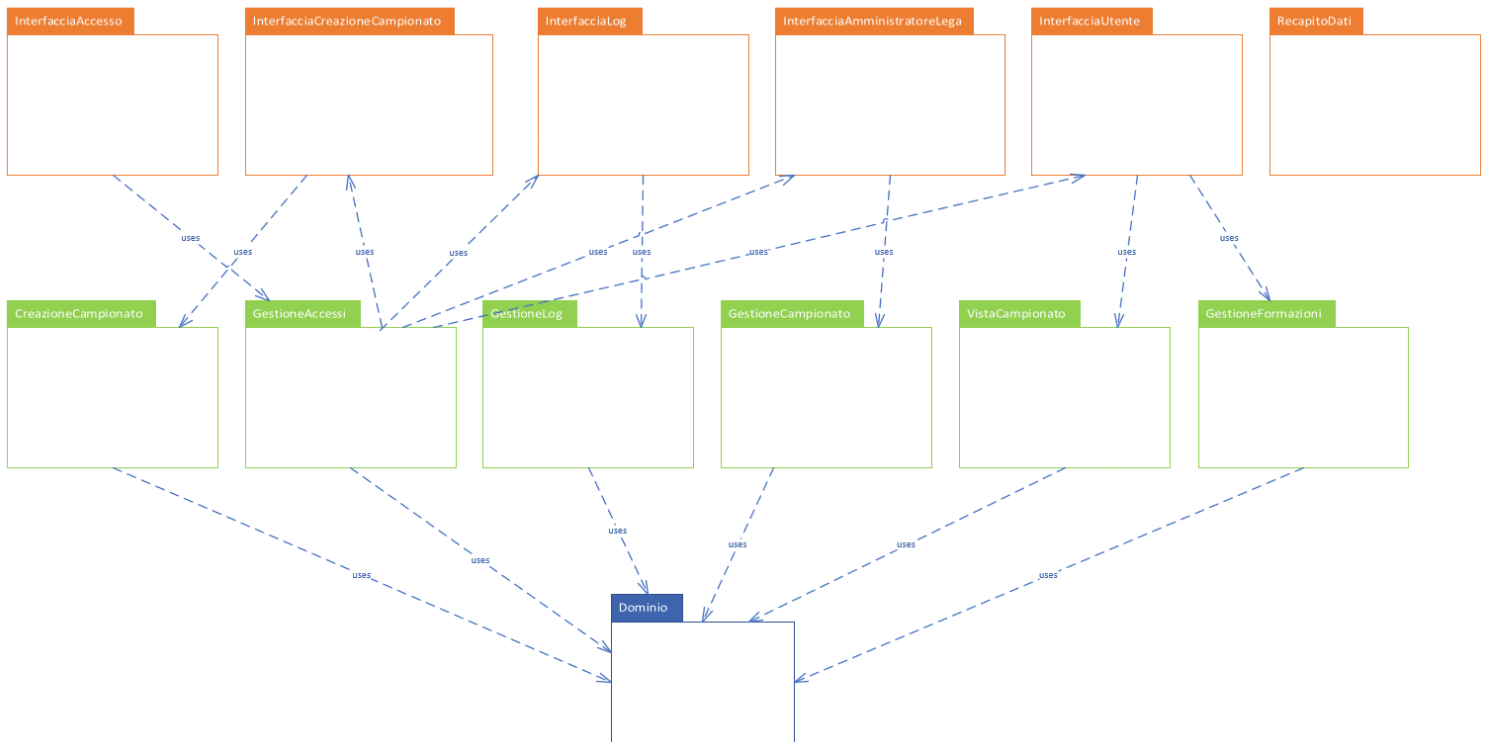
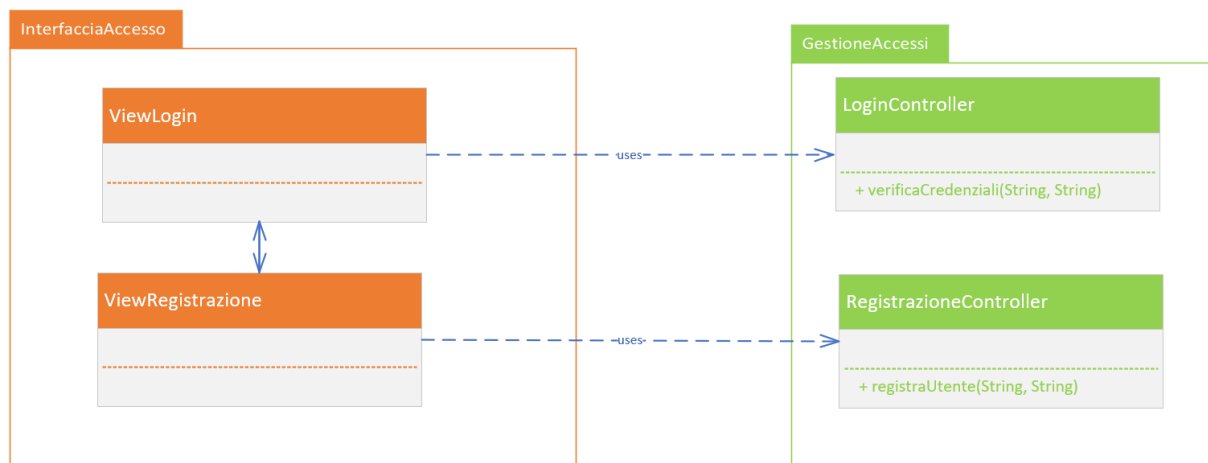


Diagramma delle classi: Dominio

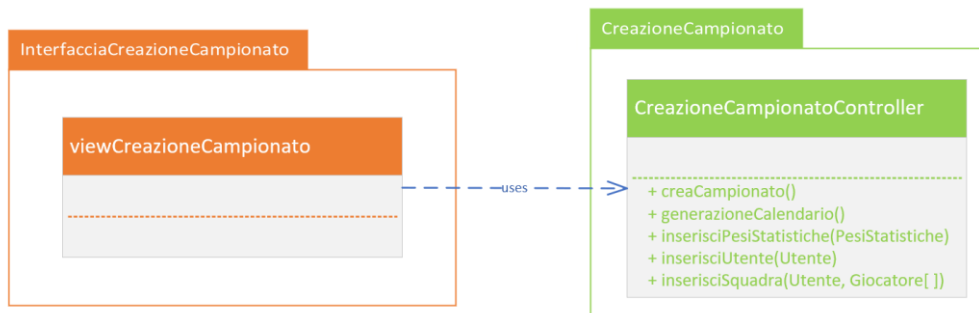
Non viene riportato il diagramma delle classi associato al package Dominio in quanto è il modello del dominio creato nella fase precedente

Diagramma delle classi: InterfacciaAccesso & GestioneAccessi



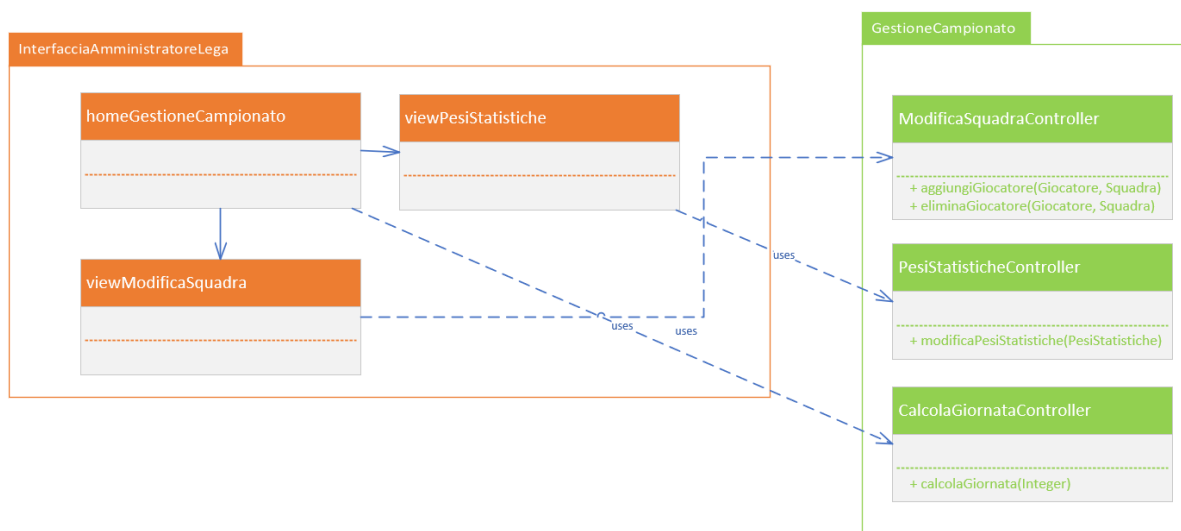
Il package `InterfacciaAccesso` permette la registrazione di nuovi utenti e la loro autenticazione. In particolare, `LoginController` si occupa di verificare le credenziali inserite in `ViewLogin`, e se corrette, reindirizzare l'utente verso la rispettiva interfaccia.

Diagramma delle classi: InterfacciaCreazioneCampionato & CreazioneCampionato



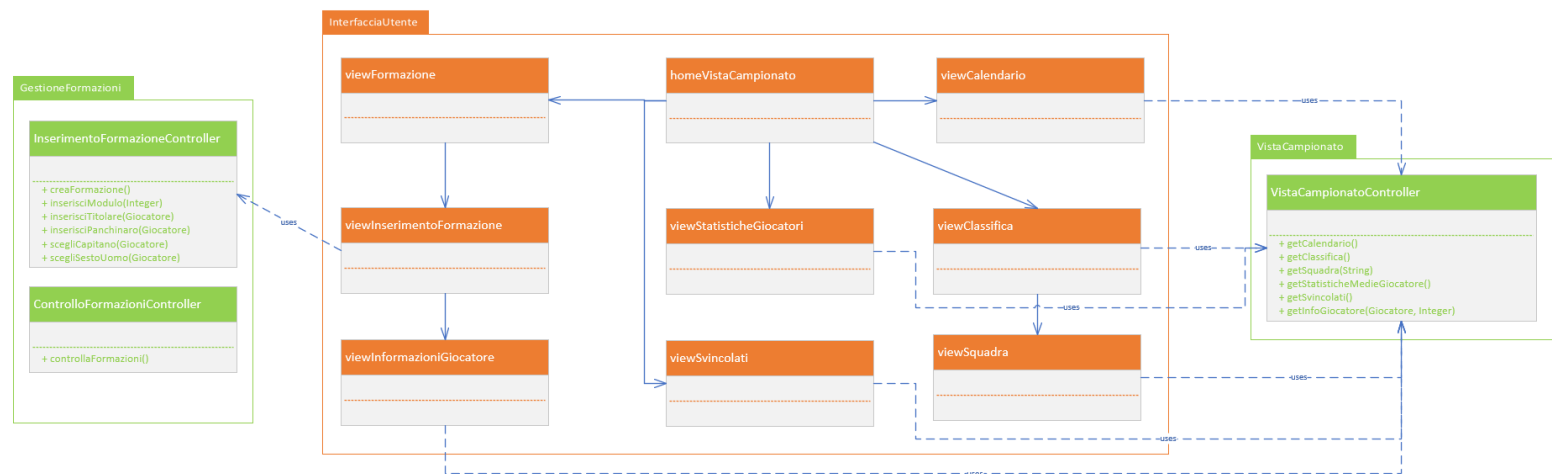
ViewCreazioneCampionato è accessibile da utenti registrati nell'applicazione, ma che non fanno parte ancora di nessuna lega. Interfacendosi con CreazioneCampionatoController permette la creazione della lega, diventandone amministratore, e l'inserimento di tutti i dati richiesti.

Diagramma delle classi: InterfacciaAmministratoreLega & GestioneCampionato



In questi due package sono racchiuse tutte le funzionalità che può svolgere l'amministratore di lega. Dalla **homeGestioneCampionato** può calcolare una giornata con **CalcolaGiornataController**, altrimenti navigando verso **viewPesiStatistiche** e **viewModificaSquadra** può o modificare i pesi delle statistiche impostati nella lega per il calcolo dei fantapunti dei giocatori, oppure modificare le squadre della lega, aggiungendo o togliendo giocatori.

Diagramma delle classi: GestioneFormazioni, InterfacciaUtente & VistaCampionato

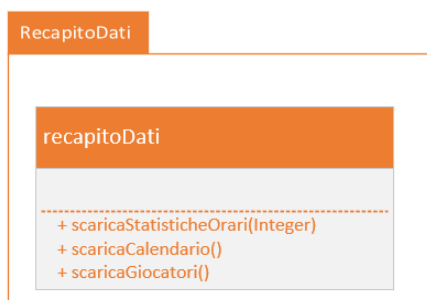


Il package `InterfacciaUtente` racchiude tutte le funzionalità che riguardano l'utente. La maggior parte di questo sono funzionalità di visualizzazione, svolte da `VistaCampionatoController`.

L'utente ha come compito fondamentale quello di inserire la formazione ad ogni giornata. In `viewFormazione` ha la possibilità di visualizzare la formazione inserita per la giornata successiva, se ne ha già inserita una. In ogni caso navigando in `viewInserimentoFormazione` può inserirne una usando le funzionalità di `InserimentoFormazioneController`.

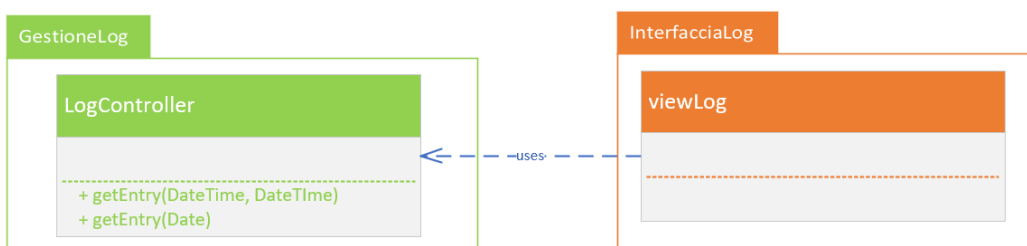
ControlloFormazioniController non si interfaccia con l'utente ed ha il compito di controllare che tutti gli utenti dell'applicazione abbiano inserito una formazione per il turno successivo al momento della scadenza.

Diagramma delle classi: RecapitoDati



L'applicazione si interfaccia con il sistema esterno attraverso RecapitoDati

Diagramma delle classi: GestioneLog & InterfacciaLog



Architettura Logica: Interazione

Diagramma di sequenza: Login eseguito con successo

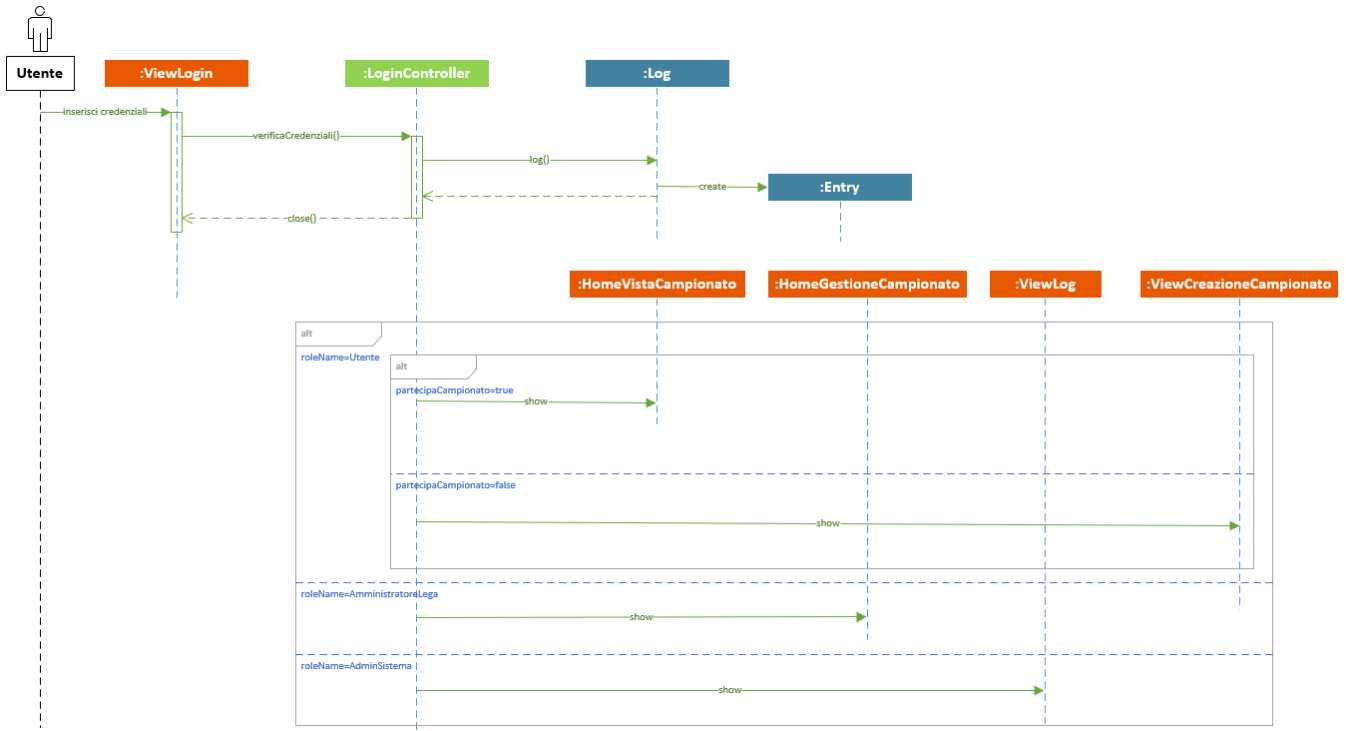
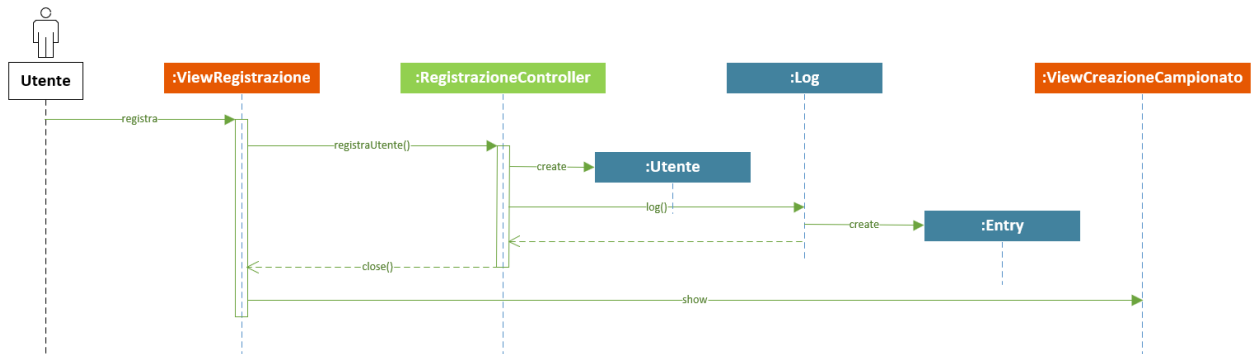


Diagramma di sequenza: Registrazione



A seguito della registrazione l'utente viene mandato su `ViewCreazioneCampionato` dove ha la possibilità di crearne uno, oppure attendere di essere aggiunto in una lega da un altro utente

Diagramma di sequenza: Creazione Campionato

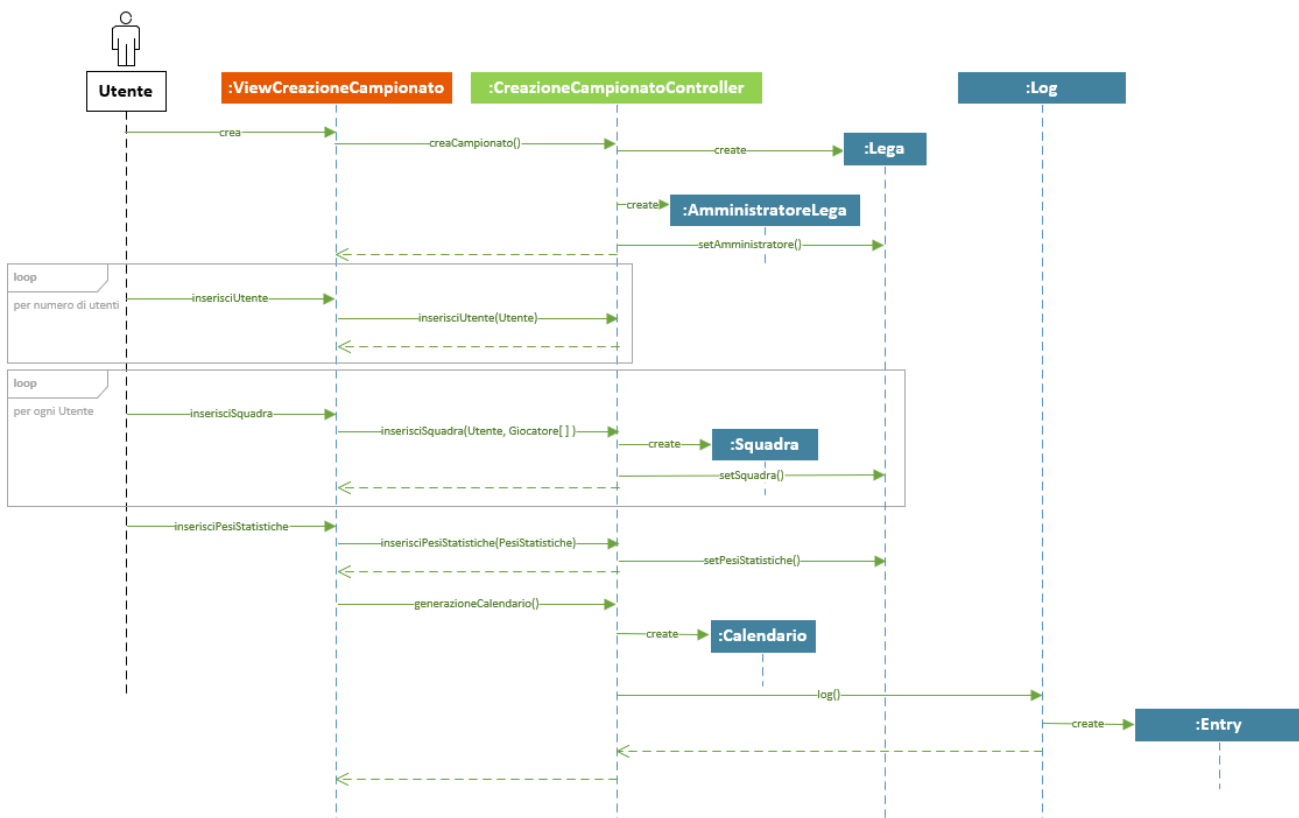


Diagramma di sequenza: Inserimento Formazione

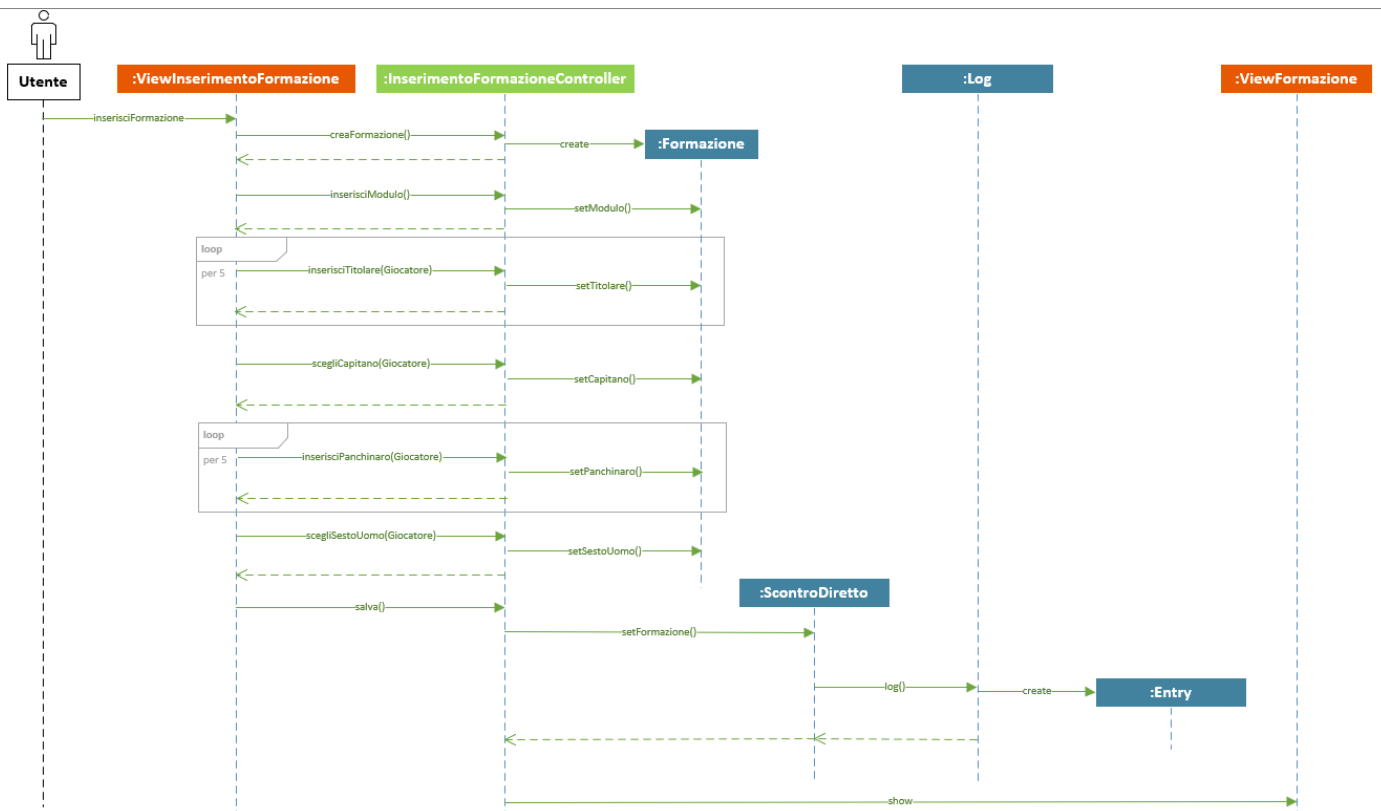


Diagramma di sequenza: Controllo Formazioni

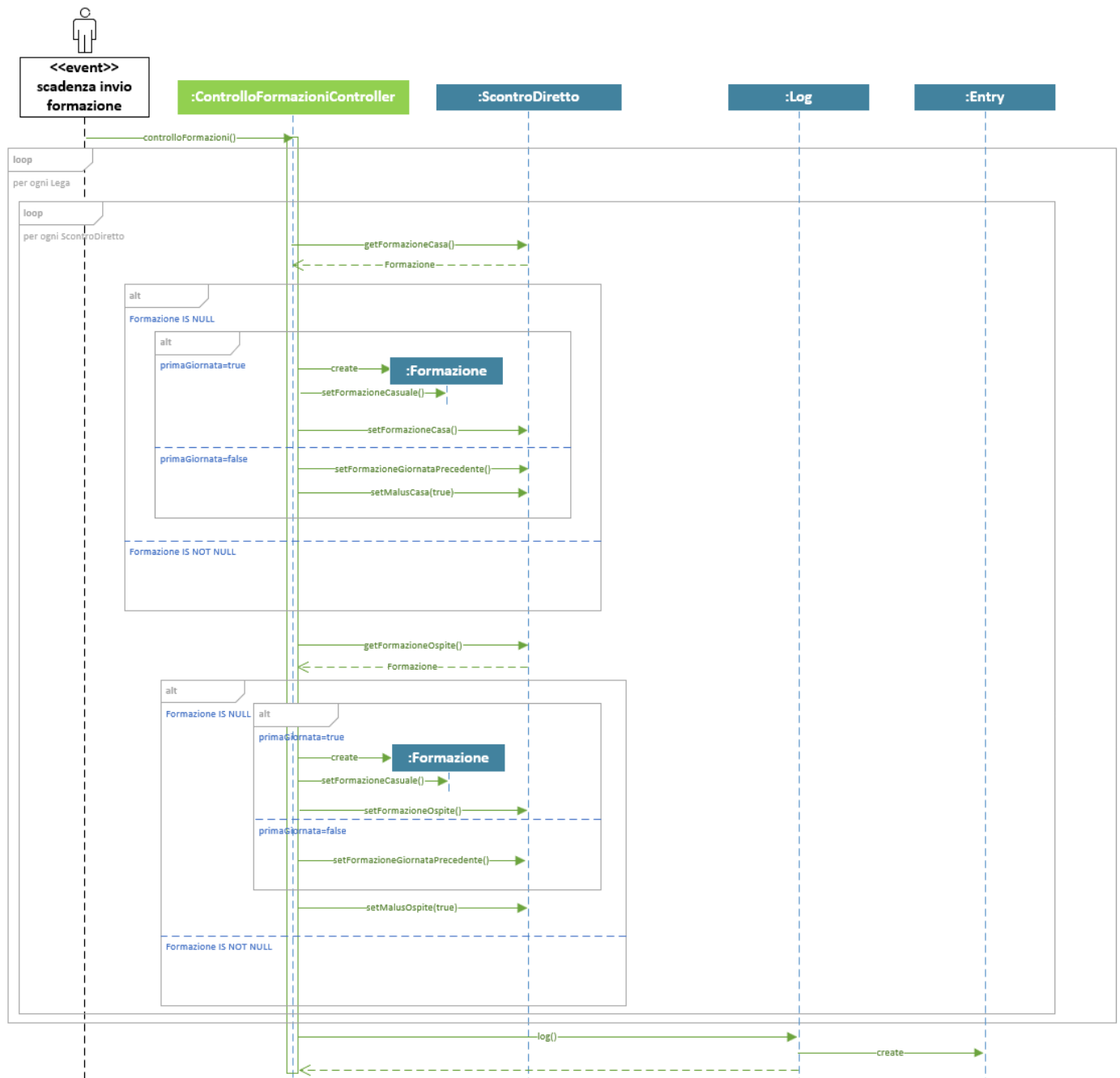


Diagramma di sequenza: Fine Giornata

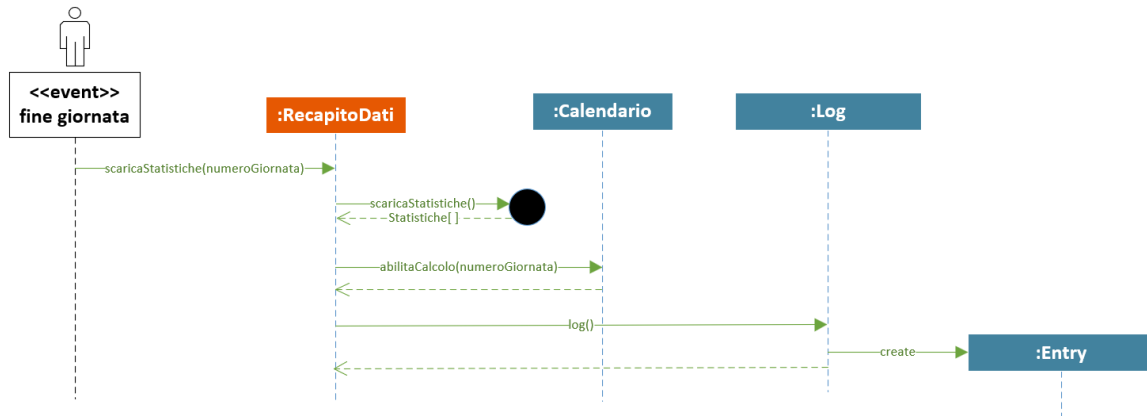


Diagramma di sequenza: Calcola Giornata

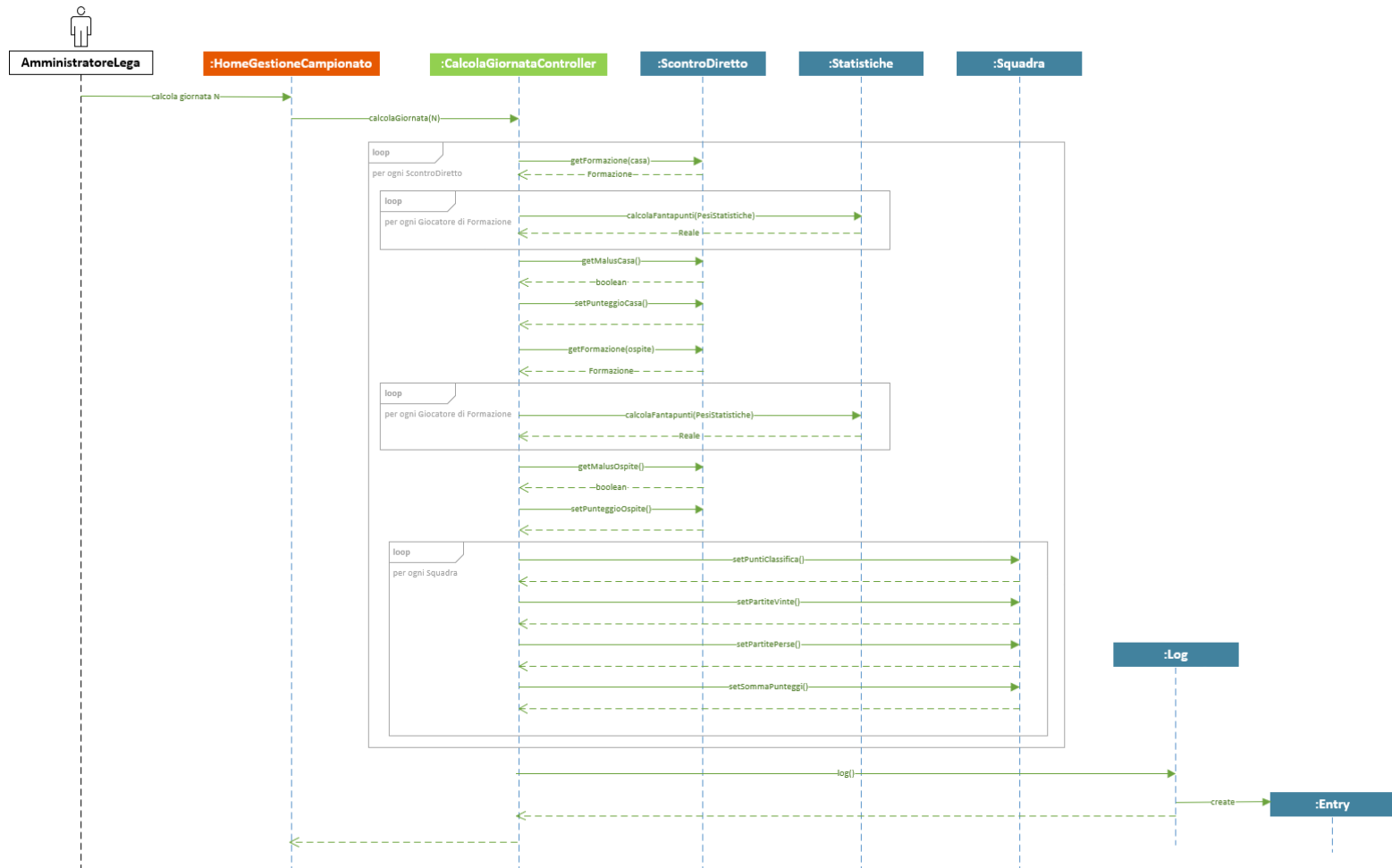


Diagramma di sequenza: Modifica Squadra

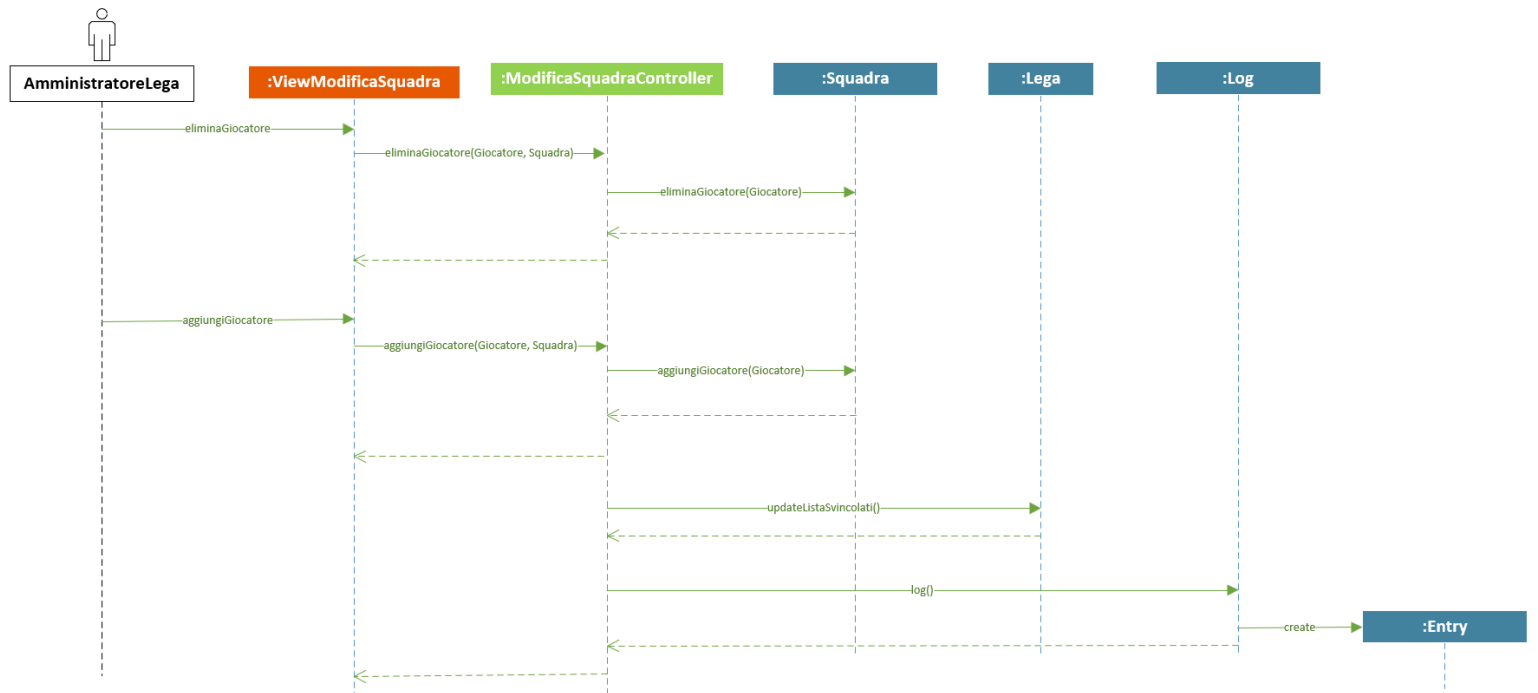
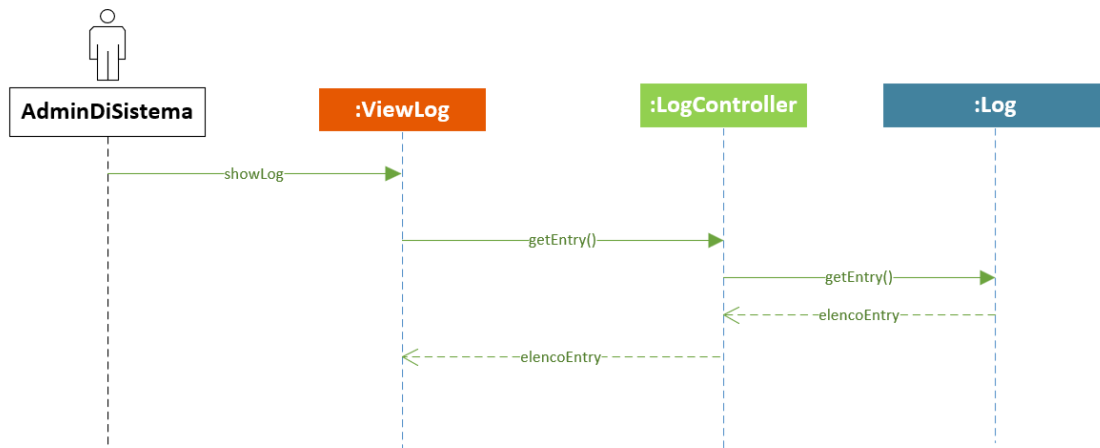


Diagramma di sequenza: Visualizzazione Log



Architettura Logica: Comportamento

In fase di analisi non si sono identificate entità così complesse da necessitare un diagramma di stato

Piano del lavoro

Package	Progetto	Sviluppo
Dominio	Cimino, Mingarelli, Spadari	Cimino
GestioneAccessi	Cimino, Mingarelli, Spadari	Cimino
GestioneLog	Cimino, Mingarelli, Spadari	Cimino
GestioneCampionato	Cimino, Mingarelli, Spadari	Mingarelli
VistaCampionato	Cimino, Mingarelli, Spadari	Cimino
GestioneFormazioni	Cimino, Mingarelli, Spadari	Mingarelli
InterfacciaUtente	Cimino, Mingarelli, Spadari	Spadari
InterfacciaAmministratoreLega	Cimino, Mingarelli, Spadari	Spadari
InterfacciaLog	Cimino, Mingarelli, Spadari	Spadari
RecapitoDati	Cimino, Mingarelli, Spadari	Mingarelli
InterfacciaCreazioneCampionato	Cimino, Mingarelli, Spadari	Spadari
InterfacciaAccesso	Cimino, Mingarelli, Spadari	Spadari
CreazioneCampionato	Cimino, Mingarelli, Spadari	Mingarelli

I tempi di rilascio previsti sono i seguenti:

- Progettazione entro fine Maggio 2023
- Sviluppo delle singole parti con collaudo unitario entro due settimane dalla fine della progettazione
- Integrazione e test dell'intero sistema entro una settimana rispetto alla fine dello sviluppo

Sviluppi futuri:

Il committente ritiene possibile che in futuro si possano aggiungere le seguenti funzionalità

- Supporto ad altri campionati reali oltre a quello LBA
- Possibilità per un utente di partecipare a più campionati, e poter essere amministratore in più di una lega per volta

Raccomandiamo al team di progettazione una particolare attenzione a queste specifiche.

Piano del collaudo

Per garantire il corretto funzionamento del sistema sono necessari una gamma di test unitari per verificare la correttezza delle singole parti. Di seguito vengono riportati quelli delle entità Giocatore, Squadra e Utente

```
[TestFixture]
public class TestSquadra {

    private Squadra squadra;

    [SetUp]
    public void SquadraSetUp() {
        Utente utente=new Utente("utenteProva");

        Giocatore[] giocatori= new Giocatore[13];
        giocatori[0]=new Giocatore("ala", "Abass Awudu", "Abass", "Virtus Bologna", new Date(1993, 1, 27));
        giocatori[1]=new Giocatore("guardia", "Marco", "Belinelli", "Virtus Bologna", new Date(1986, 3, 25));
        giocatori[2]=new Giocatore("ala", "Luigi", "Datome", "Olimpia Milano", new Date(1987, 11, 27));
        giocatori[3]=new Giocatore("ala", "Jeff", "Brooks", "Reyer Venezia", new Date(1989, 6, 12));
        giocatori[4]=new Giocatore("guardia", "Andrea", "De Nicolao", new Date(1991, 8, 21));
        giocatori[5]=new Giocatore("guardia", "Stefano", "Gentile", "Dinamo Sassari", new Date(1989, 9, 20));
        giocatori[6]=new Giocatore("centro", "Kyle", "Hines", "Olimpia Milano", new Date(1986, 9, 2));
        giocatori[7]=new Giocatore("ala", "Jamal", "Jones", "Dinamo Sassari", new Date(1993, 2, 17));
        giocatori[8]=new Giocatore("guardia", "Timothe", "Luwawu-Cabarrot", "Olimpia Milano", new Date(1995, 5, 9));
        giocatori[9]=new Giocatore("guardia", "Niccolo", "Mannion", "Virtus Bologna", new Date(2001, 3, 14));
        giocatori[10]=new Giocatore("ala", "nicolo", "Melli", "Olimpia Milano", new Date(1991, 1, 26));
        giocatori[11]=new Giocatore("centro", "Jacorey", "Williams", "Gevi Napoli basket", new Date(1994, 6, 12));
        giocatori[12]=new Giocatore("centro", "Marcus", "Lee", "Reggio-Emilia", new Date(1994, 9, 14));
        squadra=new Squadra("fantaVirtus", utente, giocatori);

    }

    [Test]
    public void TestMethod() {
        Assert.That(giocatori[7].getRuolo(), Is.EqualTo("ala"));
        Assert.That(giocatori[7].getNome(), Is.EqualTo("Jamal"));
        Assert.That(giocatori[7].getCognome(), Is.EqualTo("Jones"));
        Assert.That(giocatori[7].getSquadraLBA(), Is.EqualTo("Dinamo Sassari"));
        Assert.That(giocatori[7].getDataDiNascita(), Is.EqualTo(new Date(1993, 2, 17));

        Assert.That(utente.getUsername(), Is.EqualTo("utenteProva"));

        Assert.That(squadra.getNome(), Is.EqualTo("fantaVirtus"));
        Assert.That(squadra.getGiocatori(), Is.EqualTo(giocatori));
        Assert.That(squadra.getUtente(), Is.EqualTo(utente));
        Assert.That(squadra.getPuntiClassifica, Is.EqualTo(0));
        Assert.That(squadra.getPartiteVinte, Is.EqualTo(0));
        Assert.That(squadra.getPartitePerse, Is.EqualTo(0));
        Assert.That(squadra.getSommaPunteggi, Is.EqualTo(0));
    }
}
```

PROGETTAZIONE

PROGETTAZIONE ARCHITETTURALE

Requisiti non funzionali

Nell'Analisi del Problema (Tabella Vincoli) sono emersi tre requisiti non funzionali che impongono dei vincoli al sistema:

- Prestazioni
- Usabilità
- Protezione dei dati

Le principali funzionalità dell'applicazione saranno di fornire la possibilità all'utente di visualizzare i dati della lega, come classifica, calendario, squadre. Perciò saremo interessati ad avere buone prestazioni in termini di tempi di risposta all'interazione con l'utente, per rendergli l'esperienza d'uso piacevole. D'altra parte, essendo un'applicazione che non deve funzionare real-time, non sarà necessario garantire tempi di memorizzazione dei dati immediati.

L'applicazione si interfacerà con utenti non tecnici, quindi un requisito molto importante sarà quello di guidare gli utilizzatori con interfacce molto intuitive e facili da usare. Si richiede di porre molta attenzione su questo aspetto in fase di sviluppo del front-end.

Per la memorizzazione persistente dei dati l'applicazione si servirà di un DBMS relazionale. La maggior parte di questi dati non sono dati sensibili, per questo motivo non si ritiene opportuno la memorizzazione dei dati cifrati, ad esclusione delle password degli utenti.

La "Tabella valutazione beni" evidenzia come l'integrità dei dati sia necessaria per garantire la corretta prosecuzione delle leghe. Perciò occorre effettuare una replicazione dei dati per mantenere delle copie di backup. Dato che i dati vengono aggiornati settimanalmente si ritiene adeguata una replicazione dei dati con cadenza settimanale.

I dati possono essere manomessi, questo implica che vengano adottati dei meccanismi di controllo degli accessi e verifica dell'integrità per evitare comportamenti indesiderati. I dati in questione riguardano sia quelli memorizzati nel DBMS sia quelli in transito.

Scelta dell'architettura

L'architettura che si ritiene più adeguata per questo sistema è l'architettura Client/Server a tre livelli.

L1 – Client

L'applicazione gestisce 2 tipi di utilizzatori, gli utenti e gli amministratori di sistema, di conseguenza si è deciso di sviluppare 2 client:

- ClientUtente per le funzionalità legate all'utente
- ClientLog per le funzionalità legate all'amministratore di sistema

L2 – Server

Per quanto riguarda il lato server si è scelto di suddividere le funzionalità in 3 server:

- ServerLeghe
- ServerAccessi per la gestione del login
- ServerLog per le funzionalità di gestione dei log

L3 – Persistenza

Per la gestione dei dati persistenti dell'applicazione, si utilizza un DBMS relazionale installato su un server dedicato per ragioni di sicurezza, in quanto un eventuale violazione di un server non comprometterebbe l'altro.

Per l'interfacciamento dell'applicazione con il DB si usa la tecnica ORM, in particolare il framework Hibernate. In questo modo si rende l'applicazione indipendente dal DB sottostante.

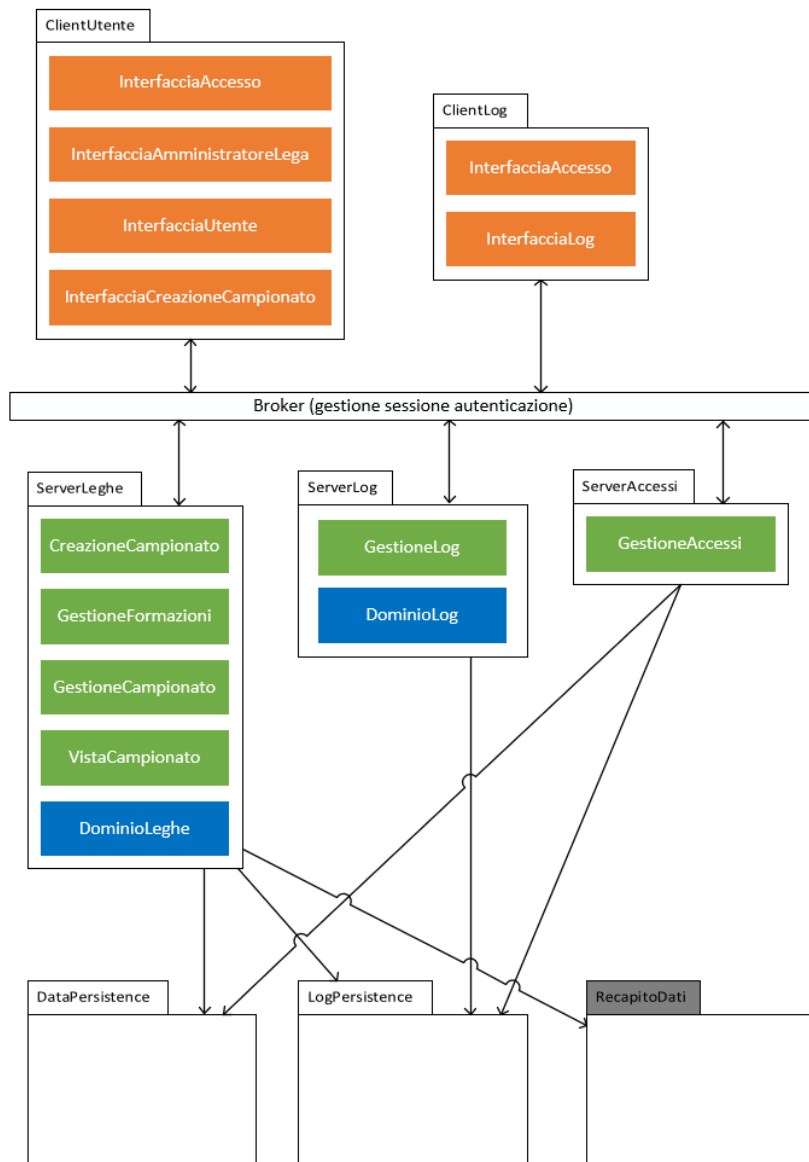
I log verranno mantenuti sul file system di un ulteriore server.

Inoltre, si ritiene opportuno l'utilizzo del Pattern Broker, a cui viene affidata la gestione delle sessioni degli utenti. Tale architettura introduce un forte disaccoppiamento tra client e server, i client non conoscono il server, ma dialogano con il broker per usufruire dei servizi del server a cui sono interessati.

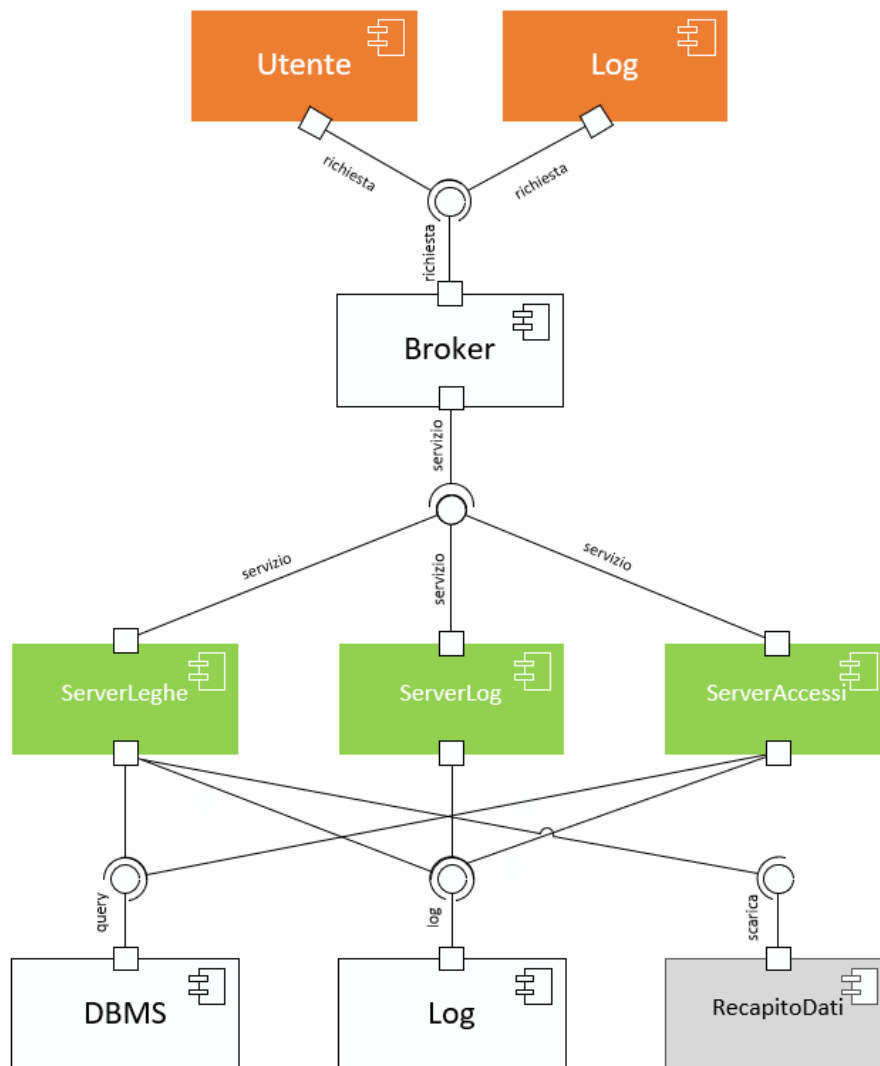
Il Pattern Broker permette anche di adottare meccanismi di load balancing, oltre a garantire la sostituzione di un server a causa di un guasto in modo trasparente per i client.

Per garantire la sicurezza delle comunicazioni, il sistema utilizzerà il protocollo crittografico TLS, che opera sopra il livello di trasporto e che garantisce autenticazione, integrità e riservatezza dei dati.

Nella figura sottostante è riportata l'architettura del sistema organizzata attraverso un **diagramma dei package**



Nella figura sottostante è riportata l'architettura del sistema organizzata attraverso un **diagramma dei componenti**



Scelte tecnologiche

A seguito di un successivo colloquio con il committente, in fase di progettazione si è scelto di implementare il sistema come un'applicazione web, in cui gli utenti accederanno all'applicazione attraverso un browser http.

La scelta è dovuta alla facilità di distribuzione dell'applicazione, in quando gli utenti non fanno parte di un gruppo ristretto e prestabilito, ma sono potenzialmente tanti ed imprevedibili. Inoltre, il committente ha espresso come volontà quella di raggiungere un pubblico quanto più ampio possibile.

L'alternativa poteva essere quella di sviluppare un'app mobile, ma in questo caso bisognava tenere in conto che per raggiungere un vasto pubblico si sarebbe dovuto sviluppare due client diversi, uno per il mondo Android e uno per il mondo iOS, il che è risultato non ottimale in seguito ad un'analisi costi/benefici.

Questa scelta tecnologica vincola il lato client ad usare le tecnologie supportate dai moderni browser web, mentre per il lato server si è scelto Java come linguaggio di programmazione OO per usufruire del framework Hibernate.

Diagramma di Dettaglio: Dominio – Lega

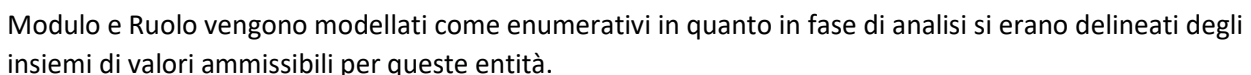
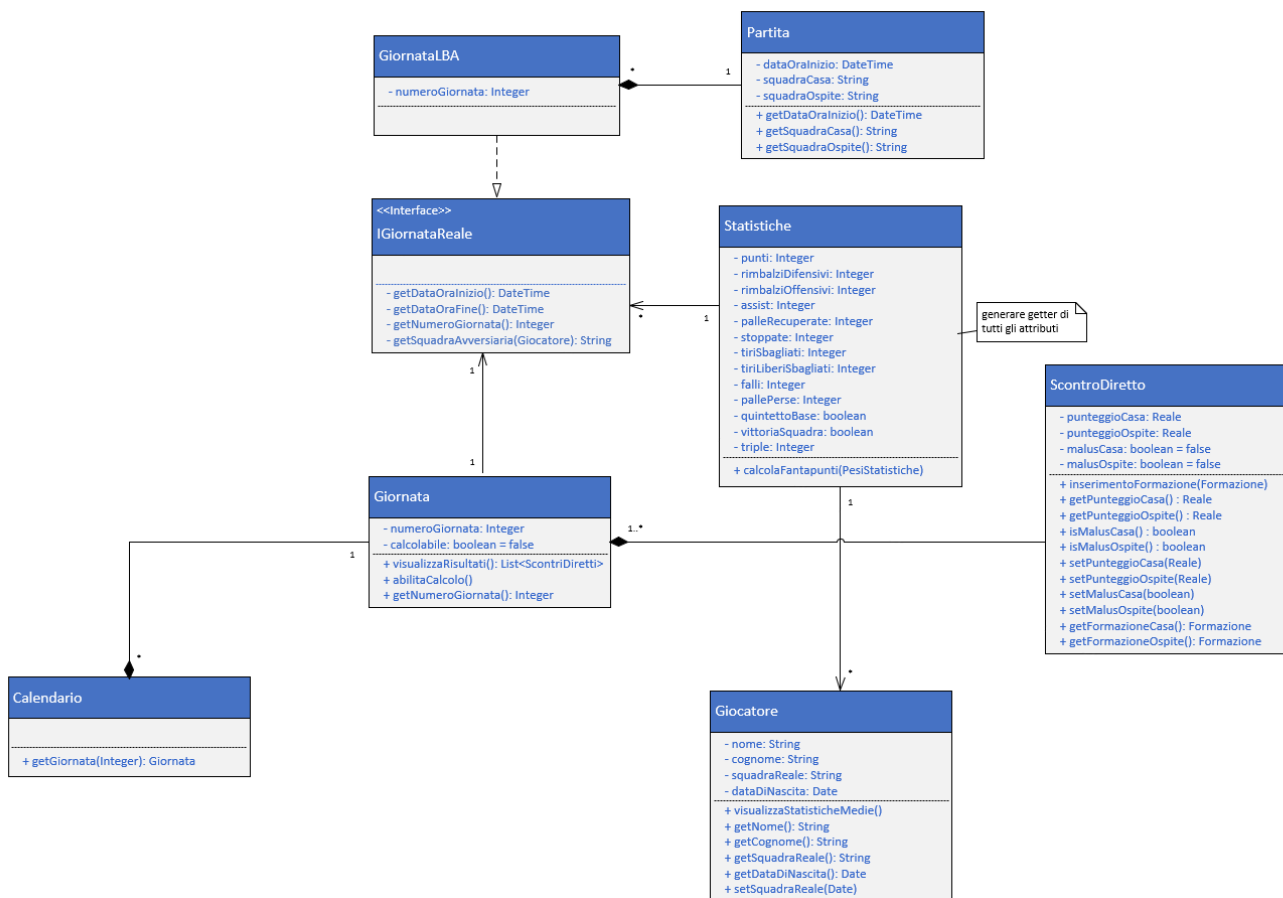


Diagramma di Dettaglio: Dominio – Calendario e Giornate



Si introduce l'interfaccia IGiornataReale per soddisfare le esigenze emerse negli sviluppi futuri, ovvero di considerare la possibilità di usare altri campionati reali diversi dal campionato LBA. L'interfaccia IGiornataReale dichiara il contratto che deve soddisfare una giornata di un campionato reale, che può essere implementata da una classe concreta diversa da GiornataLBA.

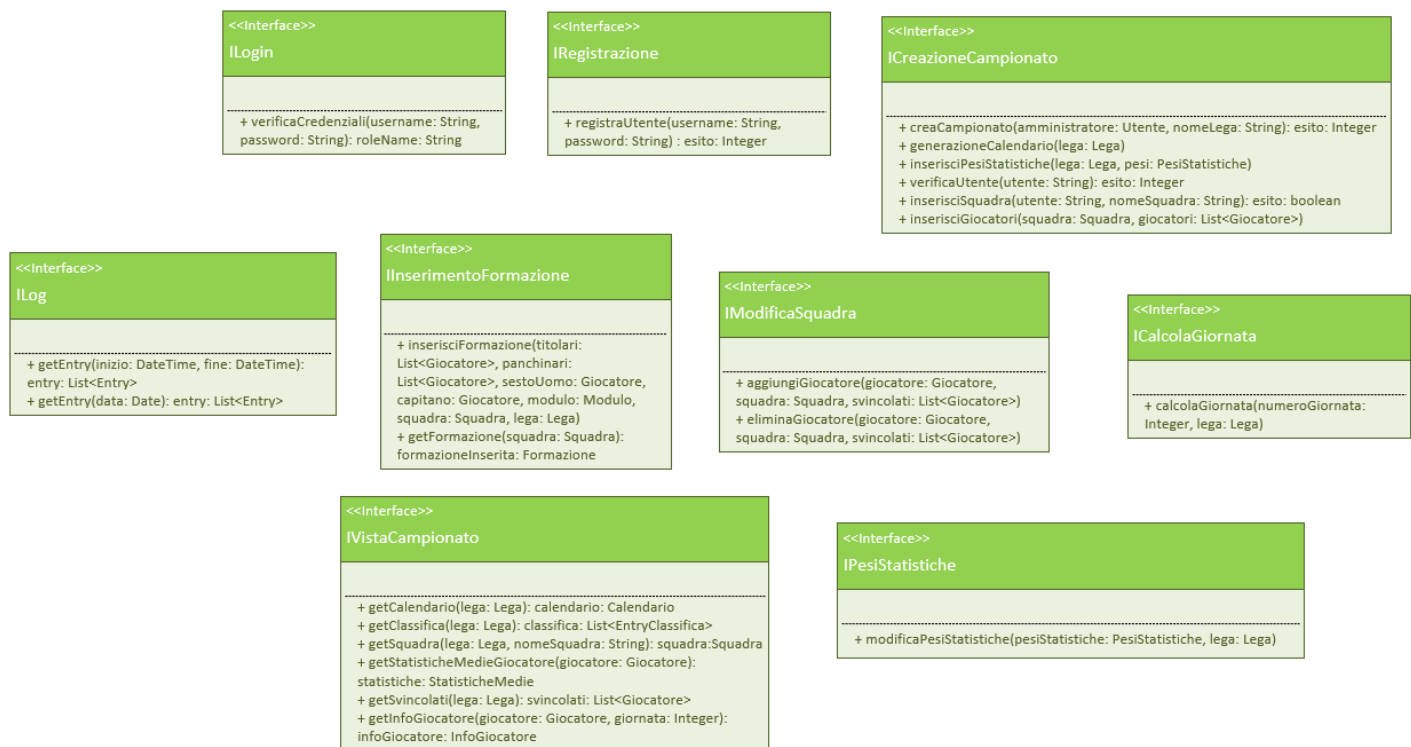
La classe Giornata rappresenta l'insieme degli scontri diretti del campionato virtuale, e mantiene il riferimento alla giornata reale corrispondente. Tale associazione permette di far corrispondere le statistiche di un giocatore in una partita del campionato reale con uno scontro diretto della lega virtuale.

La classe di associazione Statistiche viene risolta con due associazioni, con IGiornataReale e Giocatore.

Diagramma di Dettaglio: Dominio - Log

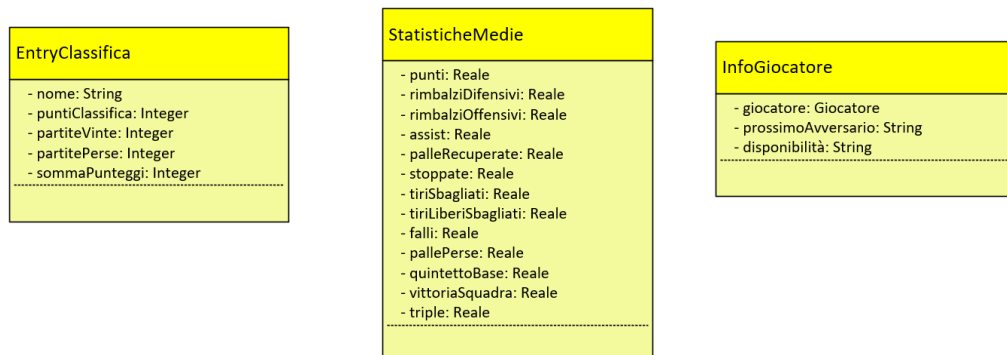


Diagramma di Dettaglio: Interfacce nei server



Si progetta uno strato di interfacce nei server che permettono di applicare “The Dependency Inversion Principle”

Diagramma di Dettaglio: Classi ausiliarie



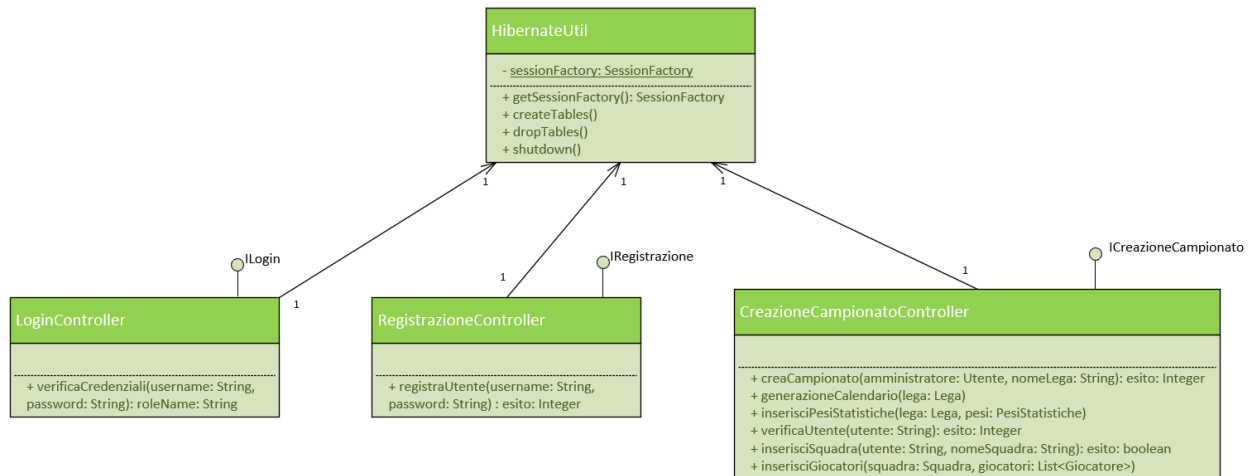
In fase di progettazione delle interfacce è emersa la necessità di disporre di una serie di classi da usare come parametri di ingresso/uscita dei metodi dei controller. Queste classi non fanno parte del dominio, bensì devono essere inserite nei package dei controller.

La classe *EntryClassifica* rappresenta una riga della classifica corrispondente ad una squadra. Nel dominio queste informazioni si trovano all’interno della classe *Squadra*, nei controller però sorge la necessità di avere una classe distinta in quanto nella classe *Squadra* è mantenuta anche la lista dei giocatori.

La classe *StatisticheMedie* rappresenta le statistiche medie di un giocatore dall’inizio della stagione. Si differenzia dalla classe *Statistiche* in quanto i campi “quintettoBase” e “vittoriaSquadra” da booleani diventano reali che rappresentano delle percentuali.

La classe *InfoGiocatore* racchiude le informazioni richieste da un utente riguardo ad un giocatore in fase di inserimento della formazione.

Diagramma di Dettaglio: HibernateUtil, Login, Registrazione, CreazioneCampionato



La classe **HibernateUtil** mantiene un riferimento ad un oggetto **SessionFactory** associato al DB. Ogni controller che ha interesse a svolgere operazioni sul DB deve avere un riferimento ad un oggetto **HibernateUtil** per poter recuperare la factory con il metodo `getSessionFactory`

Il metodo `createTables` deve essere invocato all'avvio del sistema, ed ha il compito di creare le tabelle all'interno del DB in base a quello deciso durante la progettazione della persistenza.

Diagramma di Dettaglio: InserimentoFormazione, PesiStatistiche, ModificaSquadra, VistaCampionato, CalcolaGiornata

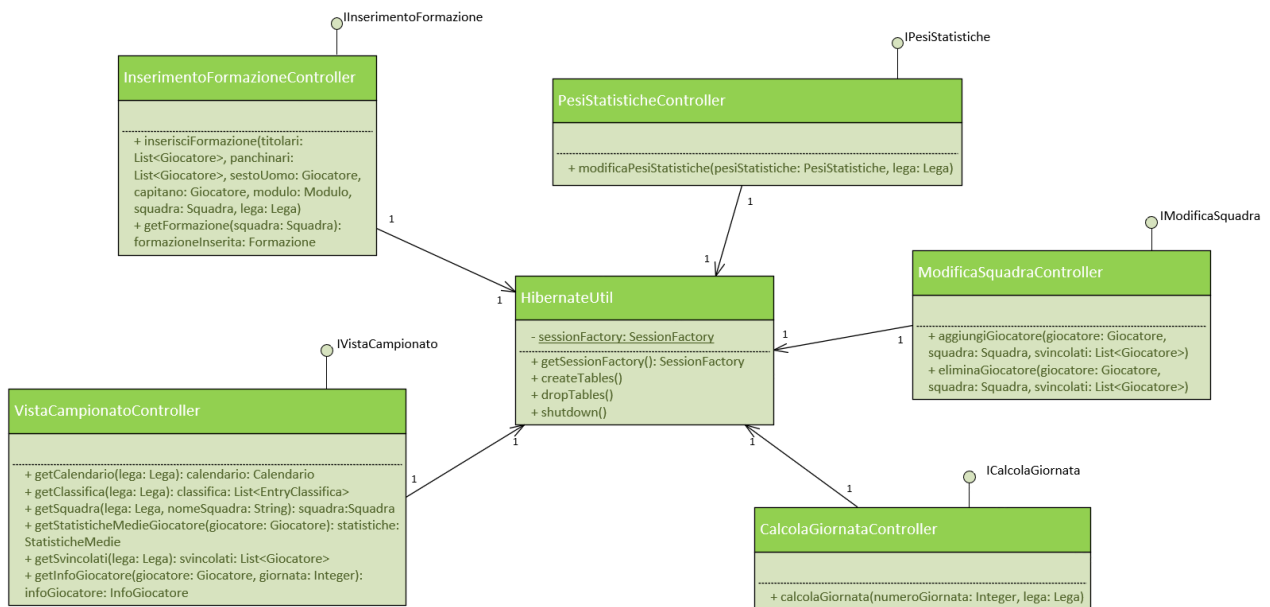
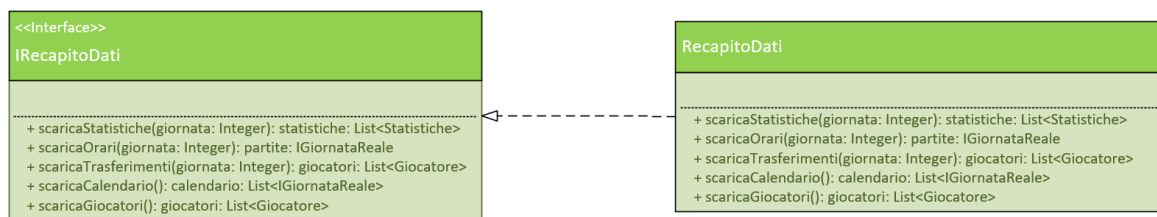


Diagramma di Dettaglio: RecapitoDati



Si utilizza il Pattern Adapter per fornire al sistema un'interfaccia standard nei confronti del sistema esterno. Il sistema esterno potrebbe cambiare nel tempo, grazie all'interfaccia **IRecapitoDati** la nostra applicazione diventa immune a questi cambiamenti, basterà infatti modificare la classe concreta che implementa l'interfaccia per ripristinare il funzionamento.

Tale interfaccia permette anche una facile estensione dell'applicazione. Sfruttando "The Open/Close Principle", cambiando la classe concreta ci si può interfacciare con diversi sistemi esterni potendo gestire anche campionati diversi da quello LBA.

La classe **RecapitoDati** svolge anche una importante funzione di parsing, in quanto riceve i dati nel formato specifico del sistema esterno, e li presenta all'applicazione sotto forma di oggetti ben formati secondo quanto dichiarato dall'interfaccia.

La classe **RecapitoDati** ha anche un importante compito di sicurezza. Durante l'analisi era emersa la necessità di garantire integrità ed autenticità sui dati trasmessi dal sistema esterno. Tali proprietà possono essere ottenute mediante l'uso del protocollo TLS ed all'autenticazione del sistema esterno mediante certificati, interamente gestito all'interno di questa classe.

I metodi *scaricaStatistiche*, *scaricaOrari* e *scaricaTrasferimenti* vengono invocati con cadenza settimanale al termine della giornata per scaricare le statistiche dei giocatori, gli orari della giornata successiva e i trasferimenti avvenuti.

ScaricaCalendario serve per richiede il calendario del campionato reale all'inizio della stagione, per poter generare i calendari delle leghe virtuali di conseguenza.

ScaricaGiocatori viene invocato all'inizio della stagione per ottenere la lista dei giocatori di tutte le squadre del campionato.

Diagramma di Dettaglio: Log

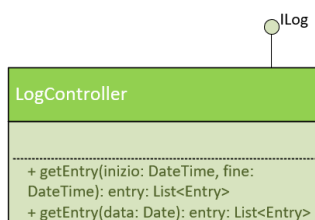
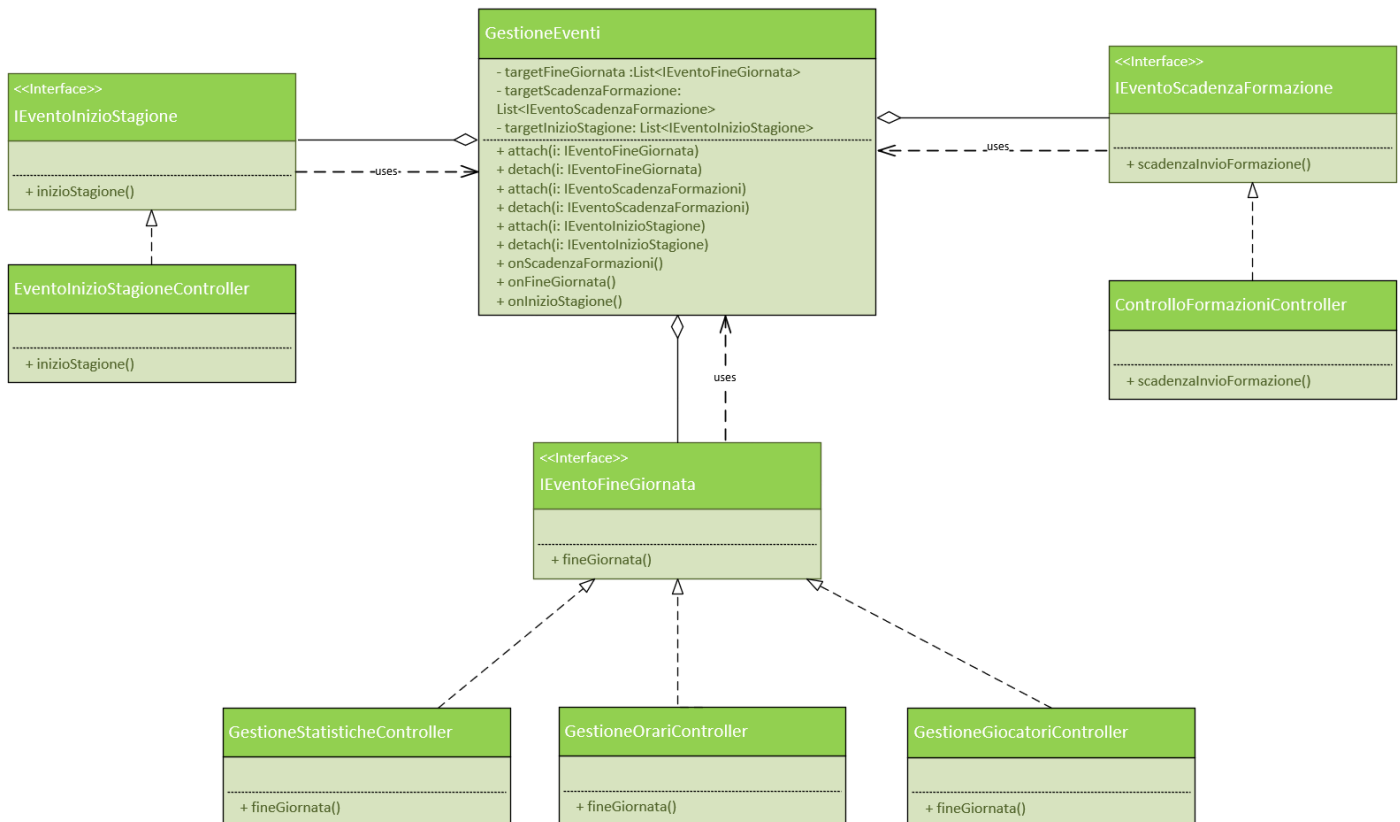


Diagramma di Dettaglio: GestioneEventi



La nostra applicazione è interessata a 3 tipi di eventi:

- Evento di inizio stagione
- Evento di fine giornata
- Evento di scadenza formazione (un'ora prima dell'inizio della giornata)

Per la gestione di questi eventi si ricorre al Pattern Observer, in cui la classe **GestioneEventi** si occupa di raccogliere le registrazioni per ogni tipo di evento, e notificare i controller registrati nel momento in cui l'evento accade.

Le sorgenti degli eventi non vengono trattate in progettazione. Si prevede che nel momento in cui viene scatenato l'evento, la sorgente invochi il metodo corrispondente di **GestioneEventi** che si occuperà di notificare gli observer registrati.

Tale pattern permette di rendere la gestione degli eventi modularizzabile: ogni controller concreto svolge un solo compito, secondo "The Single Responsibility Principle", nel momento in cui nasce la necessità di effettuare ulteriori operazioni in corrispondenza di un evento si registra un nuovo controller presso **GestioneEventi**, senza dover andare a modificare i controller già registrati.

*Anche se non viene riportato graficamente tutti i controller del diagramma di dettaglio di **GestioneEventi** hanno un riferimento ad un oggetto **HibernateUtil** e ad un oggetto **IRecapitoDati**, ad eccezione di **ControlloFormazioniController** che ha solo il riferimento ad **HibernateUtil**.*

InterfacciaAccesso

```

classDiagram
    class ViewLogin {
        +verificaCredenziali(username: String, password: String)
        +registrazione()
    }
    class ViewRegistrazione {
        +registraUtente(username: String, password: String)
        +login()
    }
    ViewLogin <--> "1" ViewRegistrazione
  
```

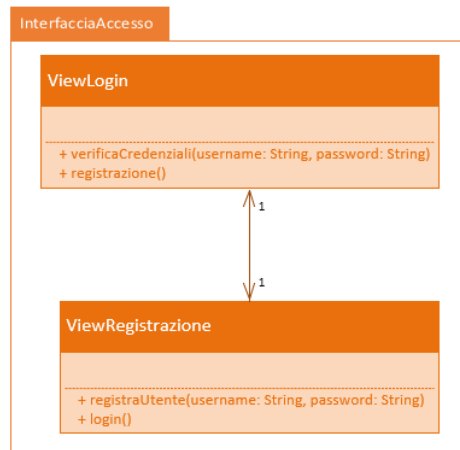


Diagramma di Dettaglio: View per la creazione del campionato



Queste sono le view per lo scenario della creazione di un nuovo campionato. Un utente dopo aver fatto il login se non partecipa a nessuna lega viene rediretto alla prima schermata, dove per entrare a fare parte di un campionato può crearlo, oppure può farsi invitare in una lega in fase di creazione.

A questo punto deve inserire il nome della lega, che deve essere diverso da tutte le leghe preesistenti, per questo motivo viene illustrato il messaggio di errore che viene mostrato nel caso il nome inserito appartenesse già ad un'altra lega.

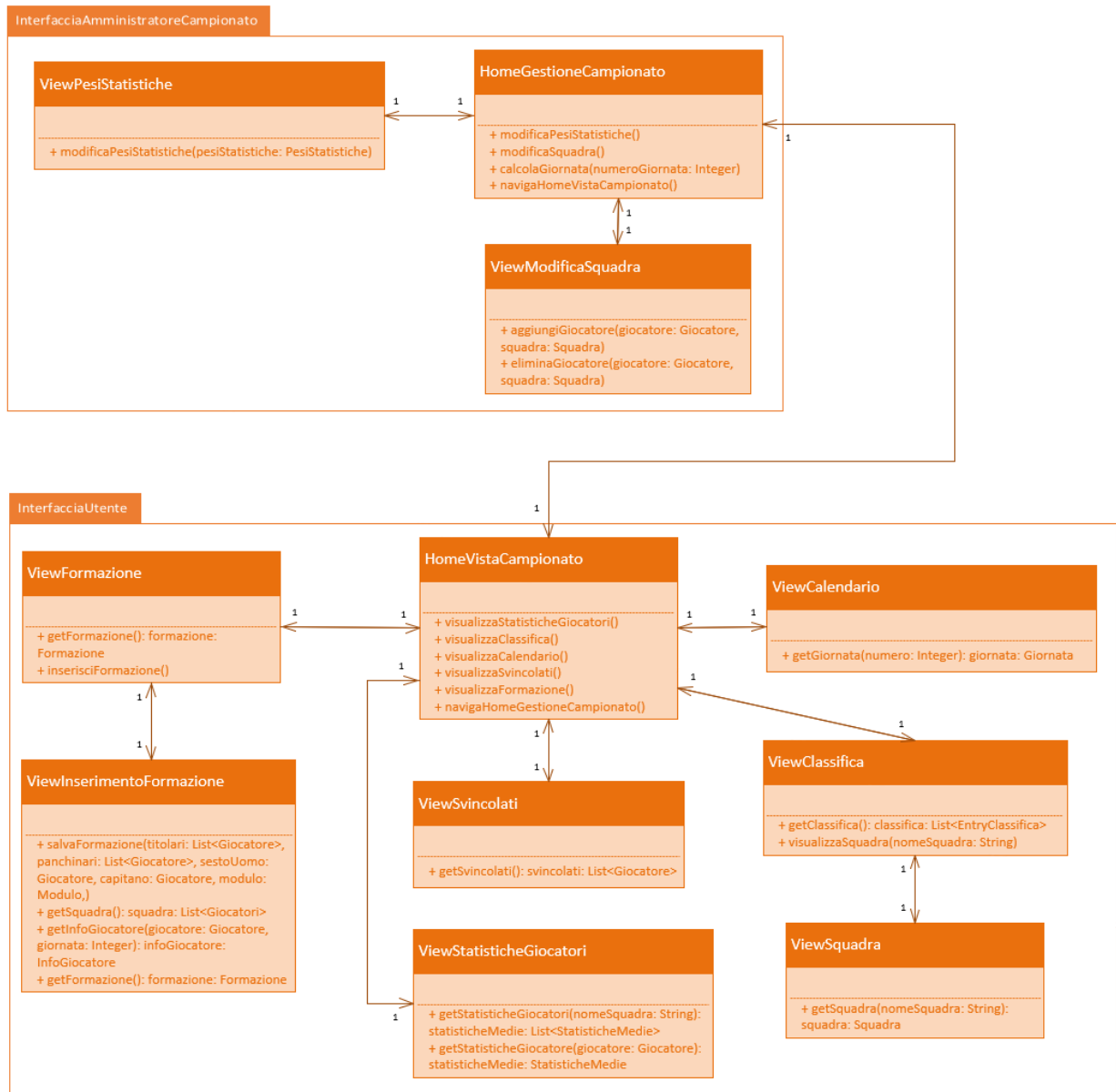
Di seguito l'inserimento degli utenti, dove in maniera simile, se non esistono viene notificato un messaggio di errore, in caso contrario invece vengono aggiunti in una entry, con la possibilità di rimuoverli cliccando sulla croce.

Infine, l'inserimento di un nome per la squadra di ogni utente, col vincolo che non si possa andare avanti se per un utente non è stato inserito niente, l'inserimento dei giocatori per ogni squadra e l'inserimento dei moltiplicatori per ogni peso statistica.

Per i giocatori basta digitare il nome sulla casella di testo e in automatico verranno proposti i nomi dei giocatori che iniziano con quelle lettere, mentre per i pesi statistiche vengono proposti i valori di default, l'utente può decidere se tenere quelli oppure cambiarne qualcuno.

Ovviamente va tenuto presente che sia le squadre, sia i pesi statistiche sono modificabili, esclusivamente dall'amministratore di lega, in qualsiasi momento.

Diagramma di Dettaglio: View disponibili agli Utenti e agli Amministratori



HomeVistaCampionato



ViewCalendario

<Back

Calendario

Giornata 1				
<				>
Casa 1	121	107	Ospite 1	
Casa 2	154	122	Ospite 2	
Casa 3	91	113	Ospite 3	
Casa 4	135	136	Ospite 4	

ViewClassifica

<Back

Classifica

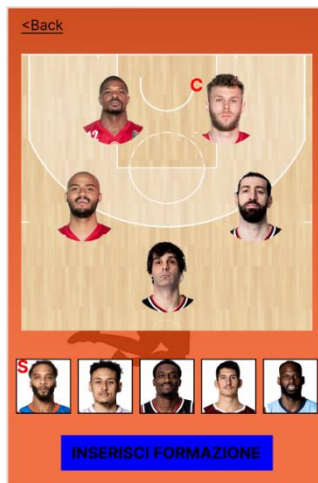
Nome squadra	V	P	somma punteggi	PC
ingegneriasoftware	3	1	654.3	6
tutti30L	2	2	562.1	4
Idisperati	2	2	557.8	4
miamiscarsa	1	3	481.5	2

ViewSquadra

<Classifica

<	ingegneriasoftware	>
	Awadu Abass	A
	Kyle Hines	C
	Milos Teodosic	G
	Adrian Banks	G
	Niccolò Mellì	C
	Shavon Shields	A
	Tornike Shengelia	A

ViewFormazione



ViewInserimentoFormazione

<Annulla

Modulo: 1-2-2 v

Invia!

The ViewInserimentoFormazione screen shows a basketball court diagram with player positions. Below the court is a row of player avatars. There are input fields for "Capitano:" and "Sesto uomo:". A blue button labeled "Invia!" is at the top right.

ViewSvincolati

<Back

Lista svincolati

Inserisci nome giocatore

	Awadu Abass	A
	Kyle Hines	C
	Milos Teodosic	G
	Adrian Banks	G
	Niccolò Mellì	C
	Shavon Shields	A
	Tornike Shengelia	A

Wi

	Jacorey Williams	C
	Derek Willis	A

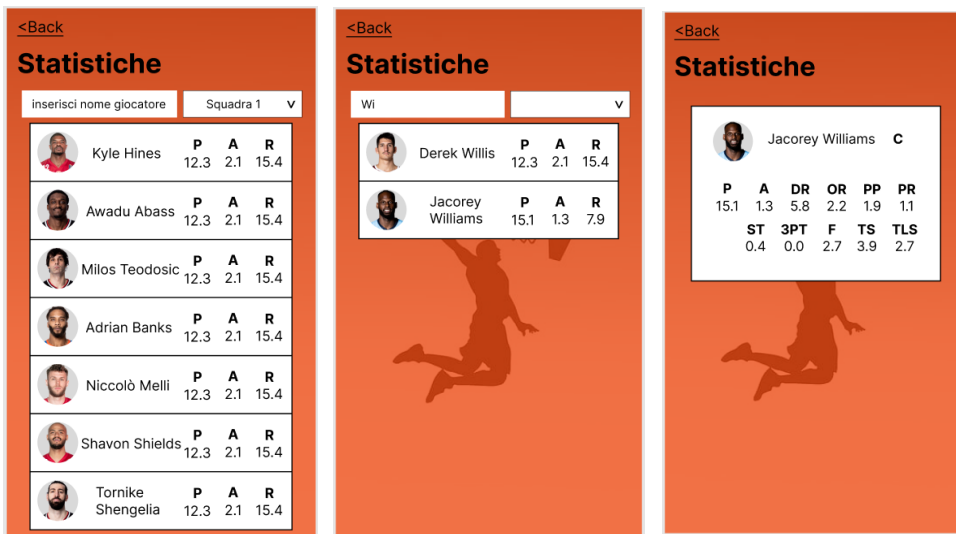
Quando un utente effettua il login e appartiene già a una lega, viene rediretto alla pagina di home del campionato, in cui può accedere alle funzionalità principali, come la visualizzazione della classifica, del calendario e l'inserimento della formazione.

In viewFormazione si può visualizzare la formazione inserita per la prossima giornata, se ne è stata già inserita una, altrimenti ne viene caricata una vuota. Attraverso i campi di inserimento, vengono proposti solamente i giocatori della propria squadra che giocano in quel ruolo.

Inserita la formazione si può salvare cliccando sul bottone "invia!". La formazione può essere modificata in qualsiasi istante fino alla scadenza, dopo la quale verrà disabilitata la possibilità di inserire la formazione per la seguente giornata.

Per quanto riguarda la viewSvincolati, vengono visualizzati tutti i giocatori svincolati della lega in cui si partecipa. Si possono scorrere, oppure se si vuole cercare un giocatore in particolare per verificare se è svincolato, lo si può cercare attraverso la barra di ricerca e verranno visualizzati solo quelli che iniziano con quelle lettere.

ViewStatistiche



Attraverso la viewStatistiche è possibile vedere le statistiche medie dei giocatori. Si possono visualizzare raggruppati per ogni squadra dei partecipanti, oppure per ricerca da campo di testo.

Viene proposta una lista di giocatori con le statistiche principali (punti, assist, rimbalzi), ma se si seleziona un giocatore, è possibile vederle tutte.

HomeGestioneCampionato

ViewPesiStatistiche

ViewModificaSquadra



Come detto in precedenza, è possibile cambiare le squadre e i pesi statistiche in qualsiasi momento, a patto che sia l'amministratore a farlo. A tal proposito bisogna aggiungere che in HomeVistaCampionato, l'icona delle impostazioni non viene visualizzata dagli utenti normali, ma solo dagli amministratori di lega.

L'accesso a questa vista, infatti, è quella che permette di entrare nella parte di gestione campionato che è accessibile solo all'amministratore.

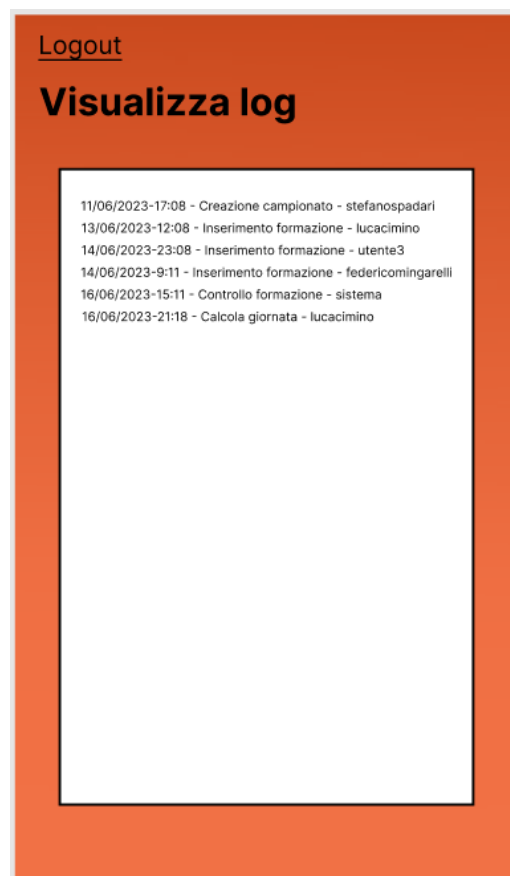
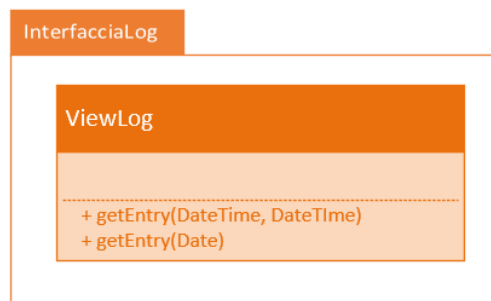
Qui è dove l'amministratore può cambiare pesi statistiche, modificare squadre oppure effettuare il calcolo della giornata.

Per la modifica dei pesi statistiche la modalità è la stessa della creazione campionato, mentre invece per quanto riguarda la modifica delle squadre, si può selezionare la squadra su cui vogliamo effettuare dei cambi, cliccando sulle croci dei giocatori essi diventano svincolati, mentre per aggiungerne dei nuovi si inserisce il nome nel campo di testo e verranno proposti col solito metodo i giocatori svincolati che rispettano la condizione. Le modifiche non vengono salvate finché non viene dato il comando "conferma".

Per terminare, si noti la parte di "calcolo giornata", sicuramente una delle funzionalità principali. Come si è detto più volte durante la spiegazione dell'app, la possibilità di calcolare la giornata può avvenire solo dopo lo scaricamento di tutte le statistiche. Perciò viene disabilitato il bottone di calcolo, se la giornata ancora non è terminata, mentre per quelle già terminate le si può selezionare attraverso il menù a tendina e cliccare su "Calcola!".

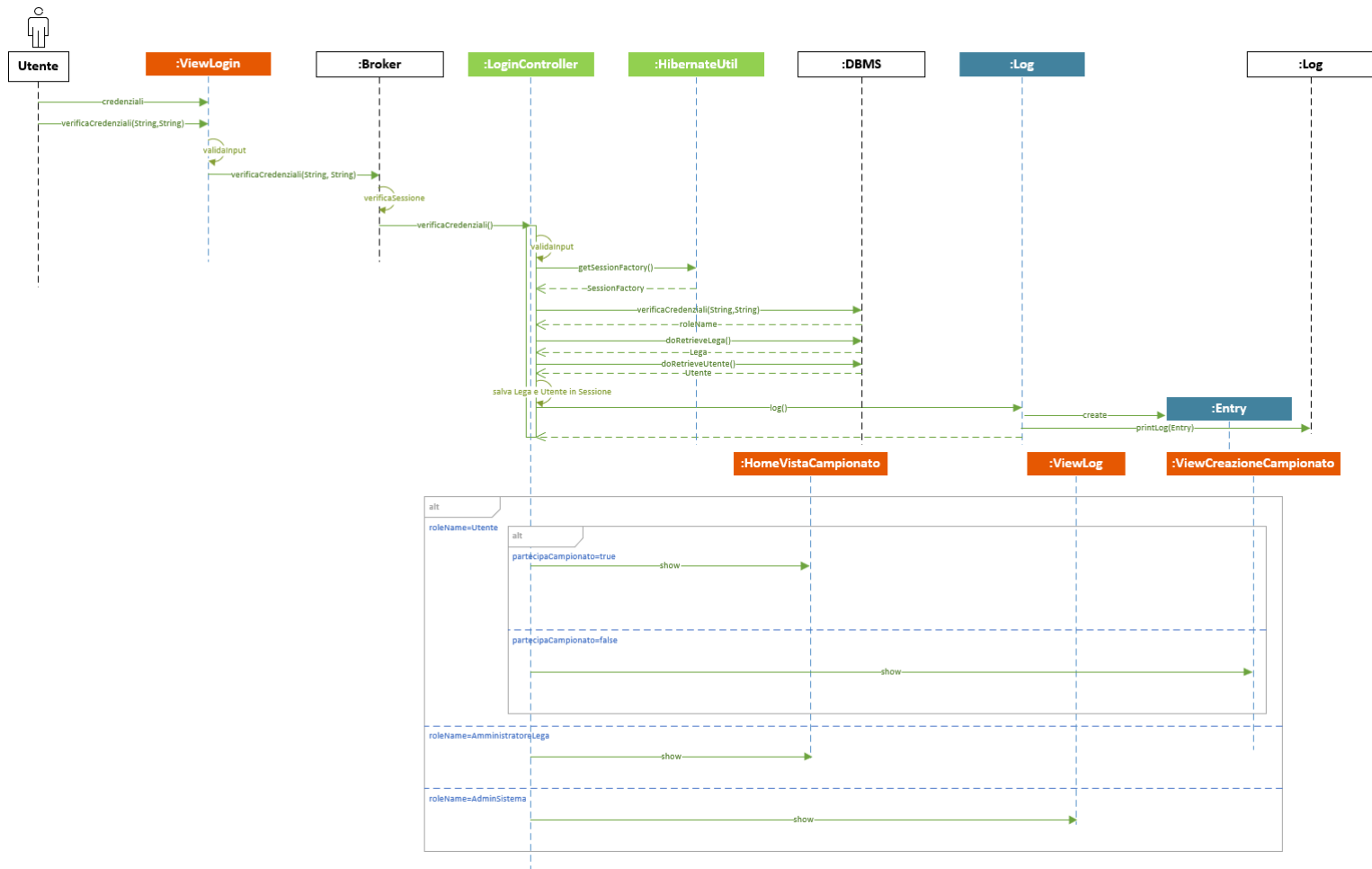
In conclusione, si voleva sottolineare il fatto che in tutti i casi in cui ci si trova in delle schermate per effettuare delle modifiche (squadre, pesi statistiche, formazioni) i bottoni di salvataggio vengono abilitati solo in seguito a delle reali modifiche, mentre se non viene apportato nessun cambiamento i bottoni rimangono "disabilitati" ed è possibile uscire dalle schermate attraverso i link di "annulla" o "back".

Diagramma di Dettaglio: View log



INTERAZIONE

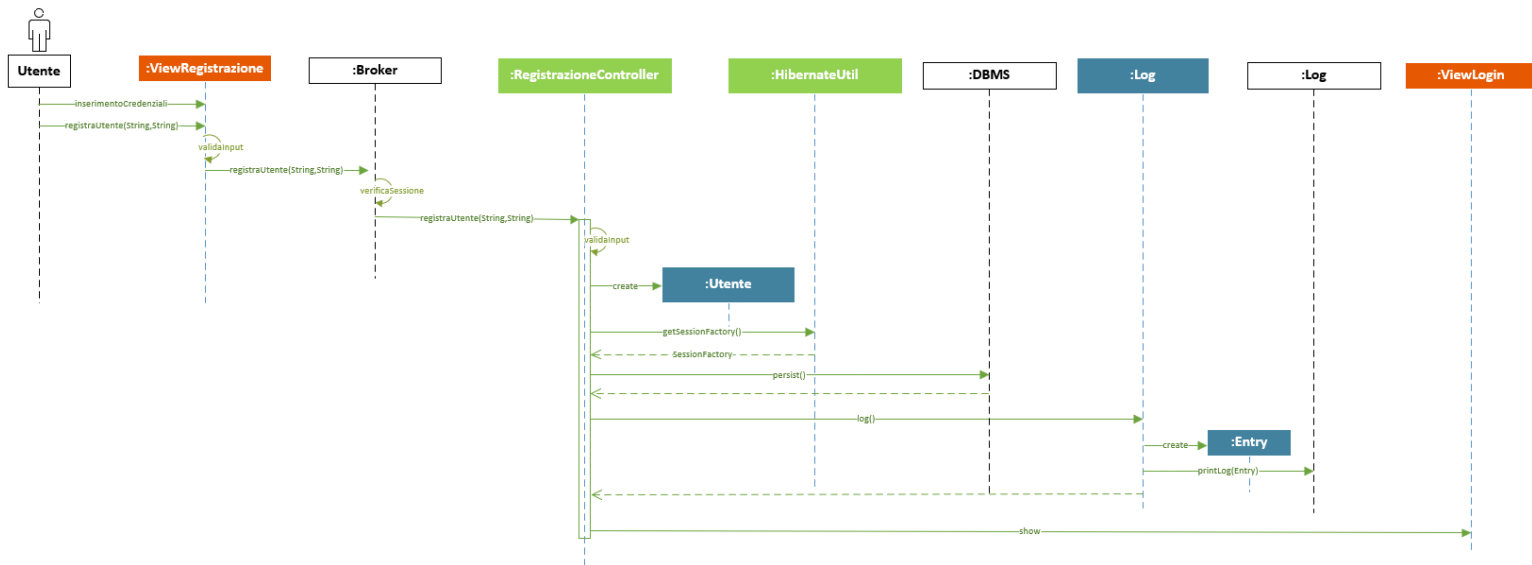
Diagramma di Sequenza: Login eseguito con successo



Tale diagramma mostra la procedura di login di un Utente nell'applicazione. Bisogna porre particolare attenzione alla validazione dell'input, effettuata sia lato client che lato server.

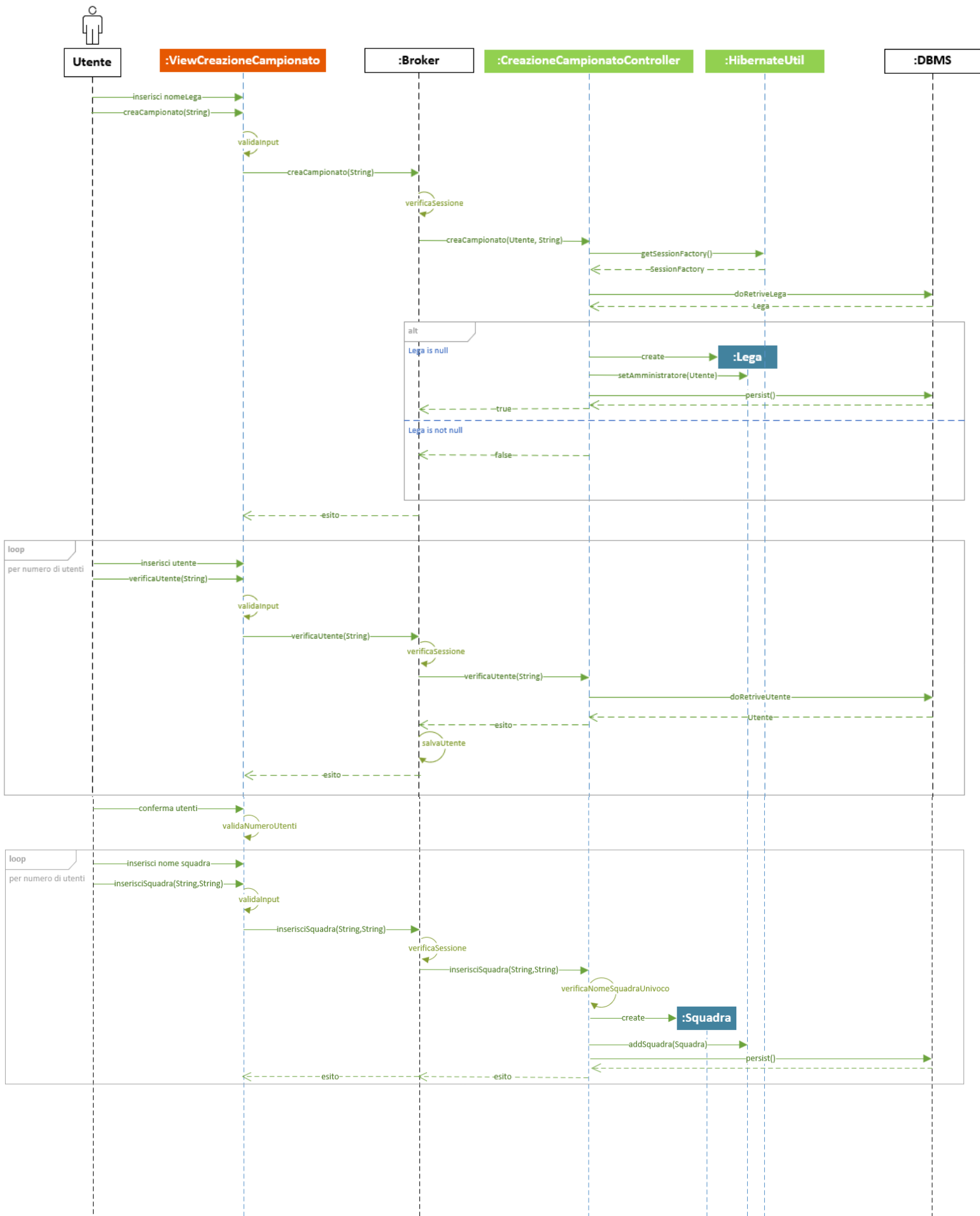
Il Pattern Broker permette l'utilizzo della sessione. In particolare, si prevede che quando l'utente supera il login con successo, all'interno della sua sessione vengano salvati i riferimenti agli oggetti Utente e Lega corrispondenti. Questo risulta particolarmente comodo quando nelle interazioni successivi del client con i server non sarà necessario recuperare ogni volta questi riferimenti in quanto già disponibili nella sessione. Sarà compito del server web recuperare dalla sessione gli oggetti necessari e invocare il metodo del controller corrispondente.

Diagramma di Sequenza: Registrazione

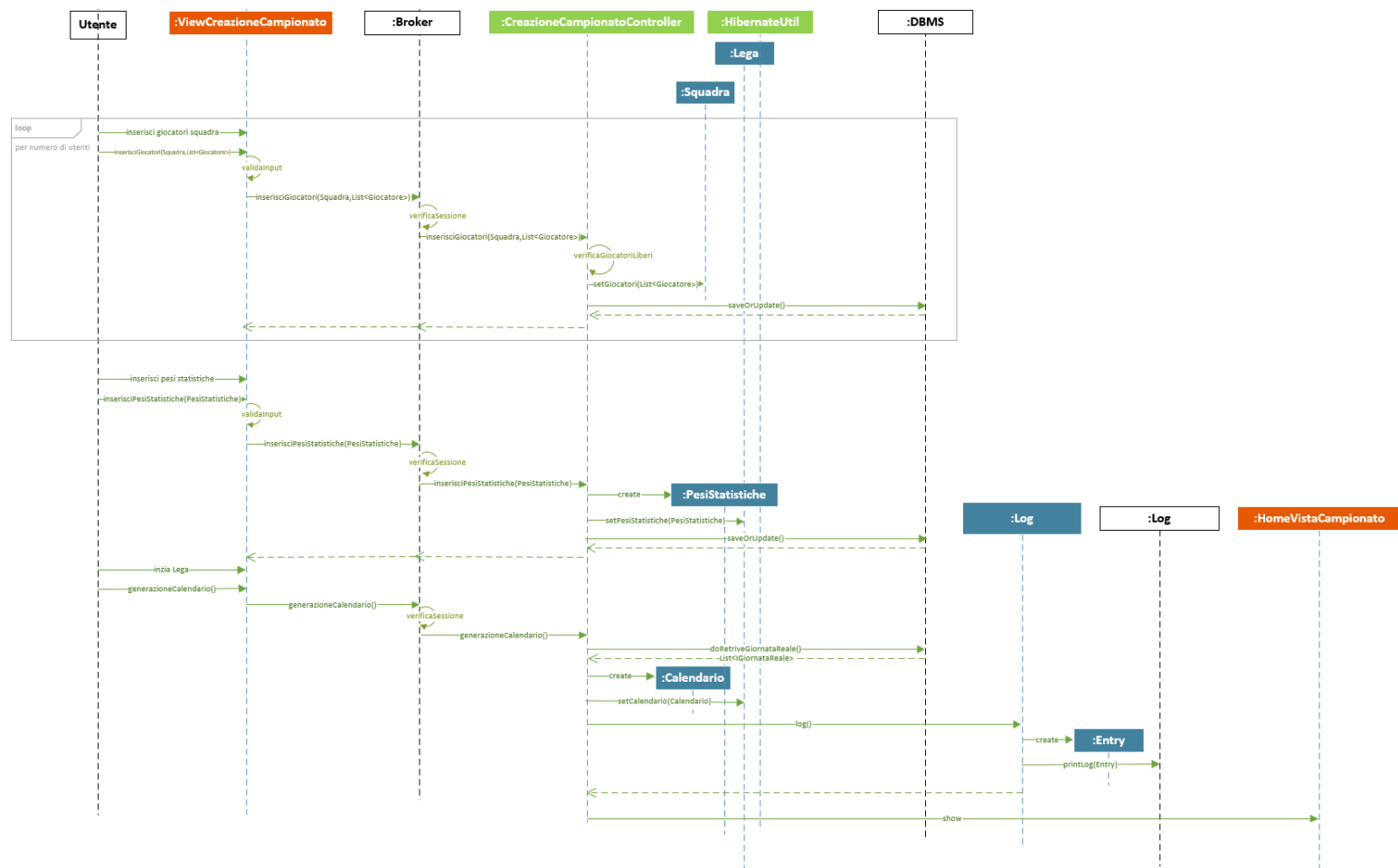


Successivamente alla sua registrazione l'utente viene indirizzato sulla ViewLogin per effettuare l'accesso al sistema

Diagramma di Sequenza: Creazione campionato



continuo...



La procedura di creazione di una lega è composta da una serie di passi che l'utente deve compiere, attraverso cui è guidato dalla ViewCreazioneCampionato. L'utente non può proseguire con il passaggio successivo se prima non ha completato con successo il precedente.

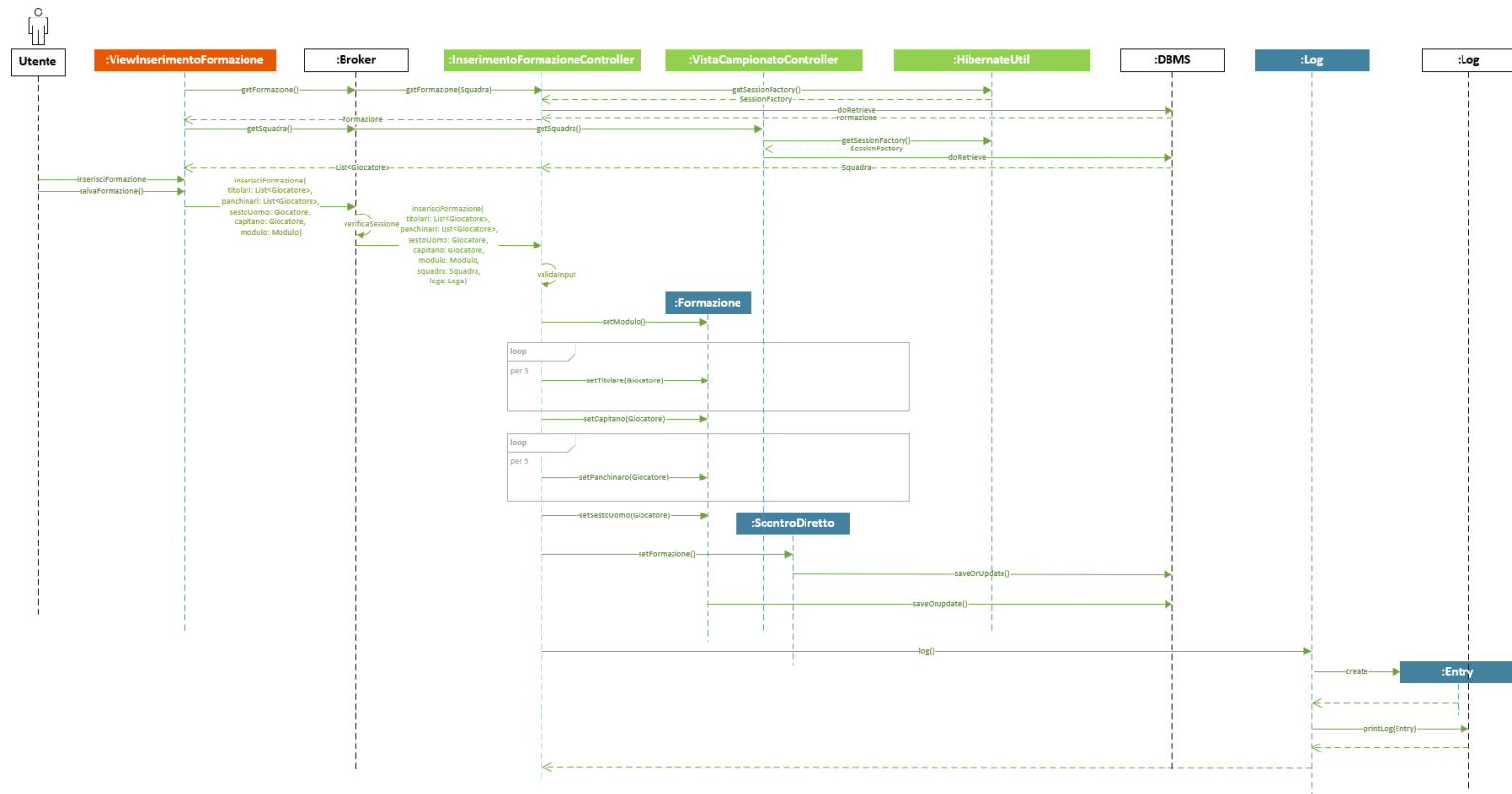
Per prima cosa viene chiesto all'utente di inserire il nome della lega che vuole creare. Il nome della lega deve essere univoco nell'applicazione, per questo motivo il metodo creaCampionato restituisce un esito sulla base di quello che ottiene con il doRetrieveLega sul DB. Finché l'utente non inserisce un nome della lega valido non può proseguire.

In seguito, viene chiesto di inserire gli utenti che comporranno la lega. I vincoli sono che devono essere minimo 2 e devono essere un numero pari. Inoltre, per ogni utente inserito bisogna verificare che esista e che non faccia già parte di nessuna lega. Il metodo verificaUtente, una volta che ha verificato un Utente lo salva nell'oggetto sessione di modo che quando ne viene inserita la squadra corrispondente possa essere direttamente associato ad essa.

Successivamente l'utente inserisce le squadre degli utenti. Il nome della squadra deve essere univoco all'interno della lega, inoltre, un giocatore può far parte di al massimo una squadra per lega.

La sequenza termina con l'inserimento dei pesi delle statistiche e la generazione del calendario della lega.

Diagramma di Sequenza: Inserimento formazione

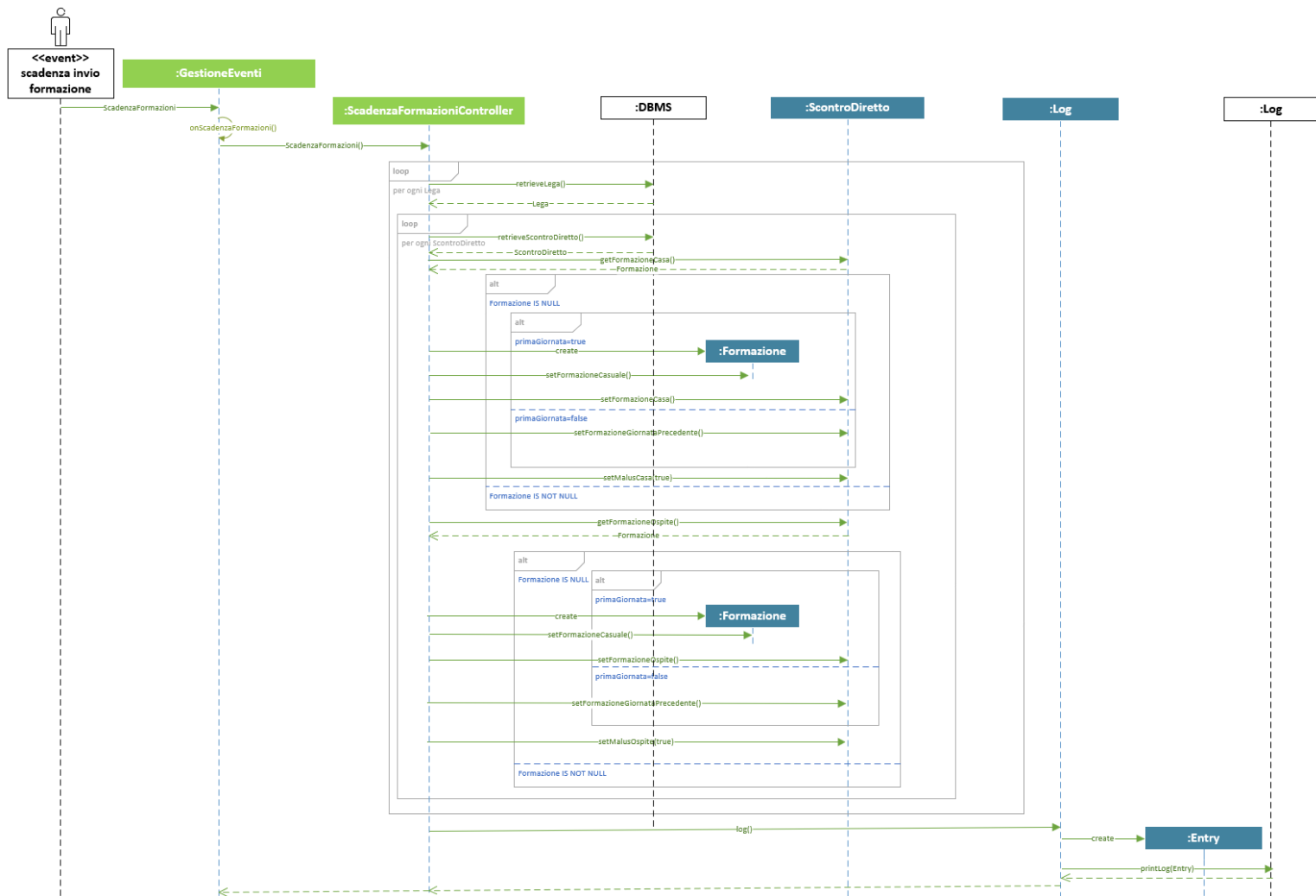


Per prima cosa la `ViewInserimentoFormazione` controlla se l'utente ha già inserito una formazione per la giornata successiva, e in tal caso la mostra.

Successivamente l'utente può modificare la formazione o inserirne una nuova da capo. Tutte le modifiche temporanee vengono mantenute lato client e vengono inviate al server solamente al momento della conferma di inserimento della formazione.

Nel diagramma di sequenza viene mostrato il caso in cui ci sia già una formazione inserita, viene recuperata dal DB, modificata e resa nuovamente persistente. Nel caso non ci sia già una formazione inserita è necessario istanziarne una nuova.

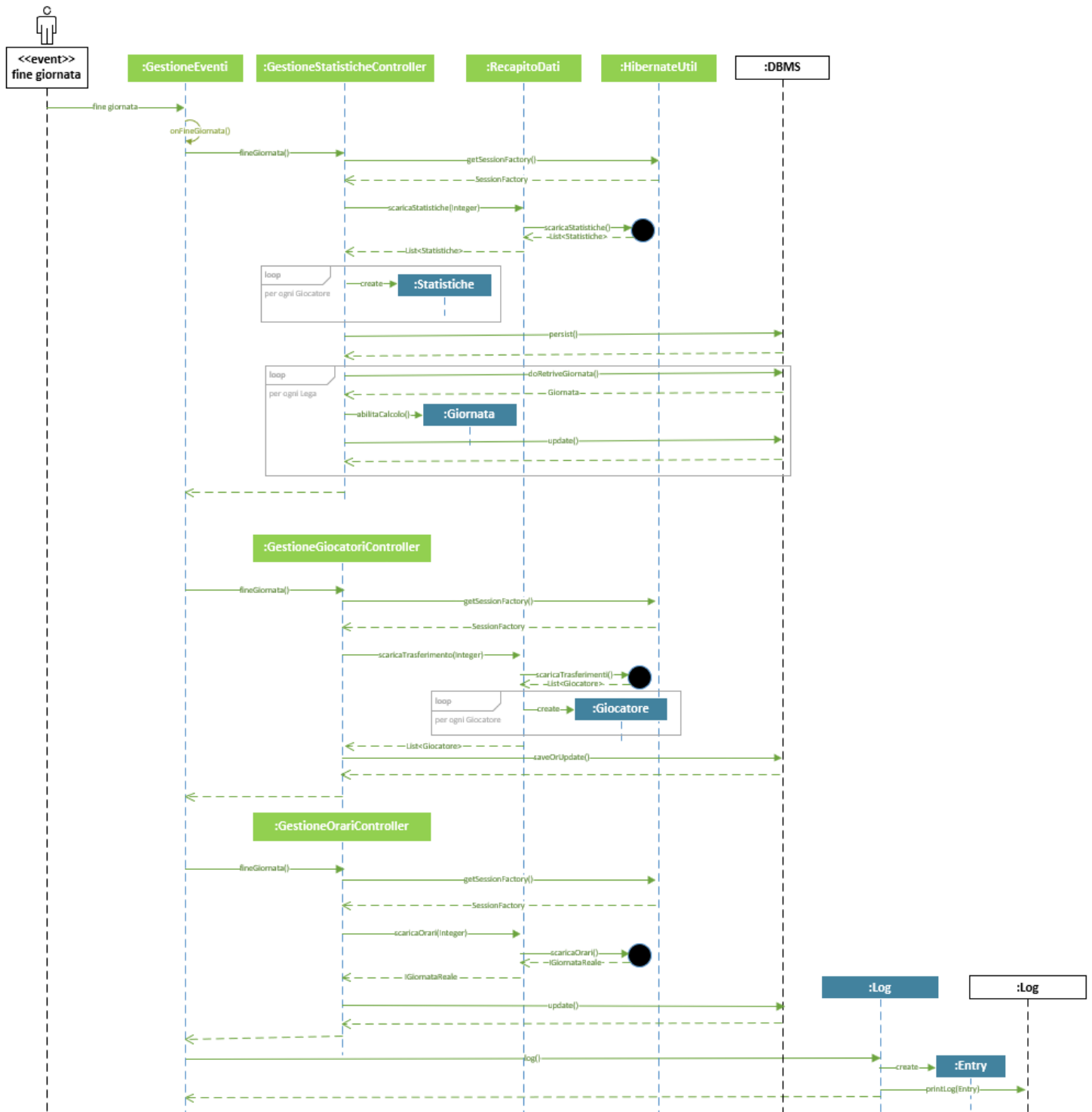
Diagramma di Sequenza: Controllo formazioni



L'evento <<scadenza invio formazioni>> invoca il metodo onScadenzaFormazioni di GestioneEventi, che si occupa di notificare i controller registrati.

Il metodo ScadenzaFormazioni controlla che tutti gli utenti di tutte le leghe dell'applicazione abbiano inserito una formazione valida per la giornata. In caso contrario provvede ad inserirgli quella schierata la giornata precedente, assegnando però un malus all'utente. Il caso in cui non venga inserita una formazione alla prima giornata viene gestito con la generazione di una formazione casuale.

Diagramma di Sequenza: Fine giornata



L'evento fine giornata scatena in sequenza l'invocazione di tutti i controller registrati per quell'evento in GestioneEventi.

In particolare, i tre controller di questo diagramma si occupano di scaricare dati rilevati dal sistema esterno interfacciandosi con il controller RecapitoDati.

Diagramma di Sequenza: Calcola giornata

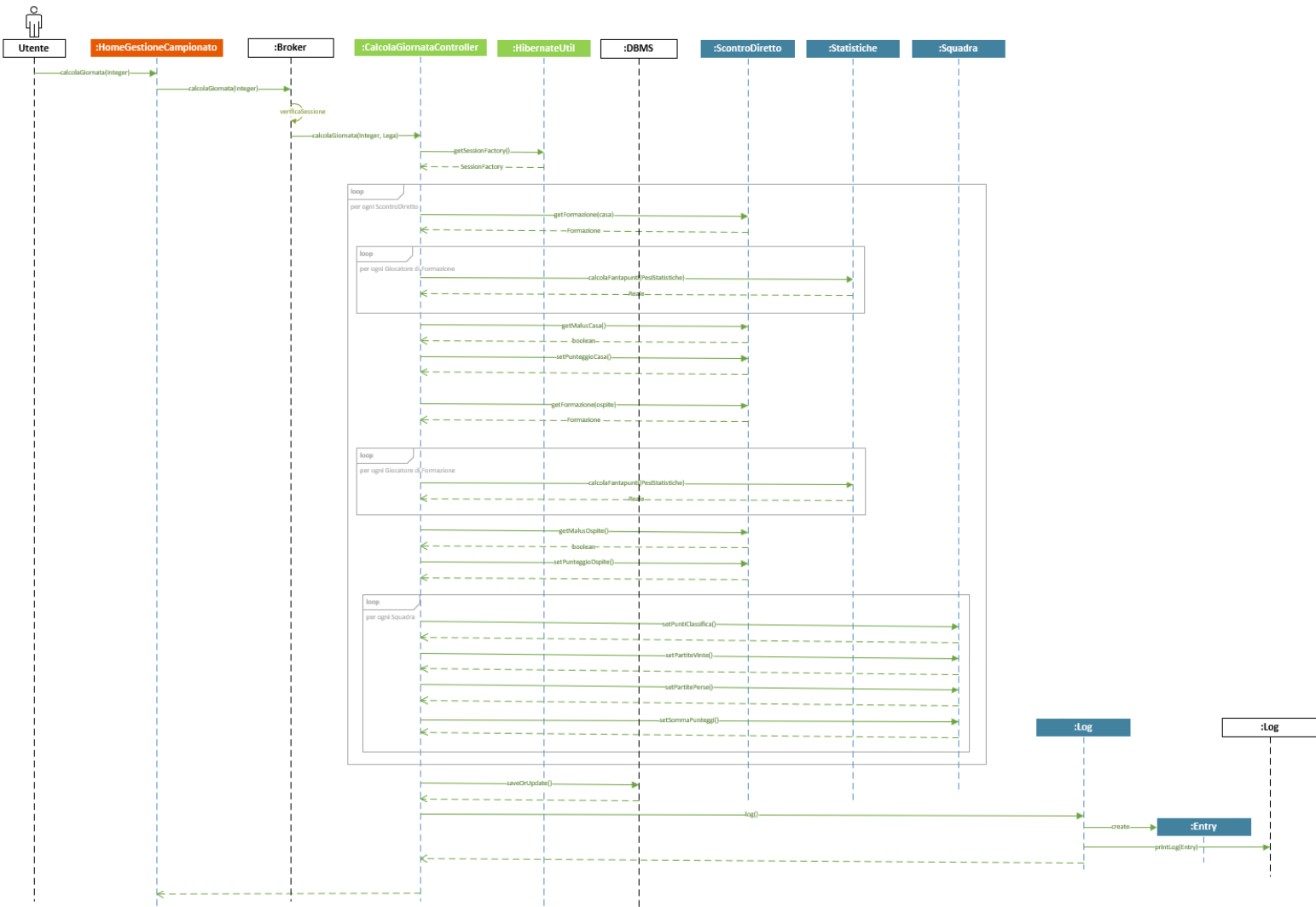
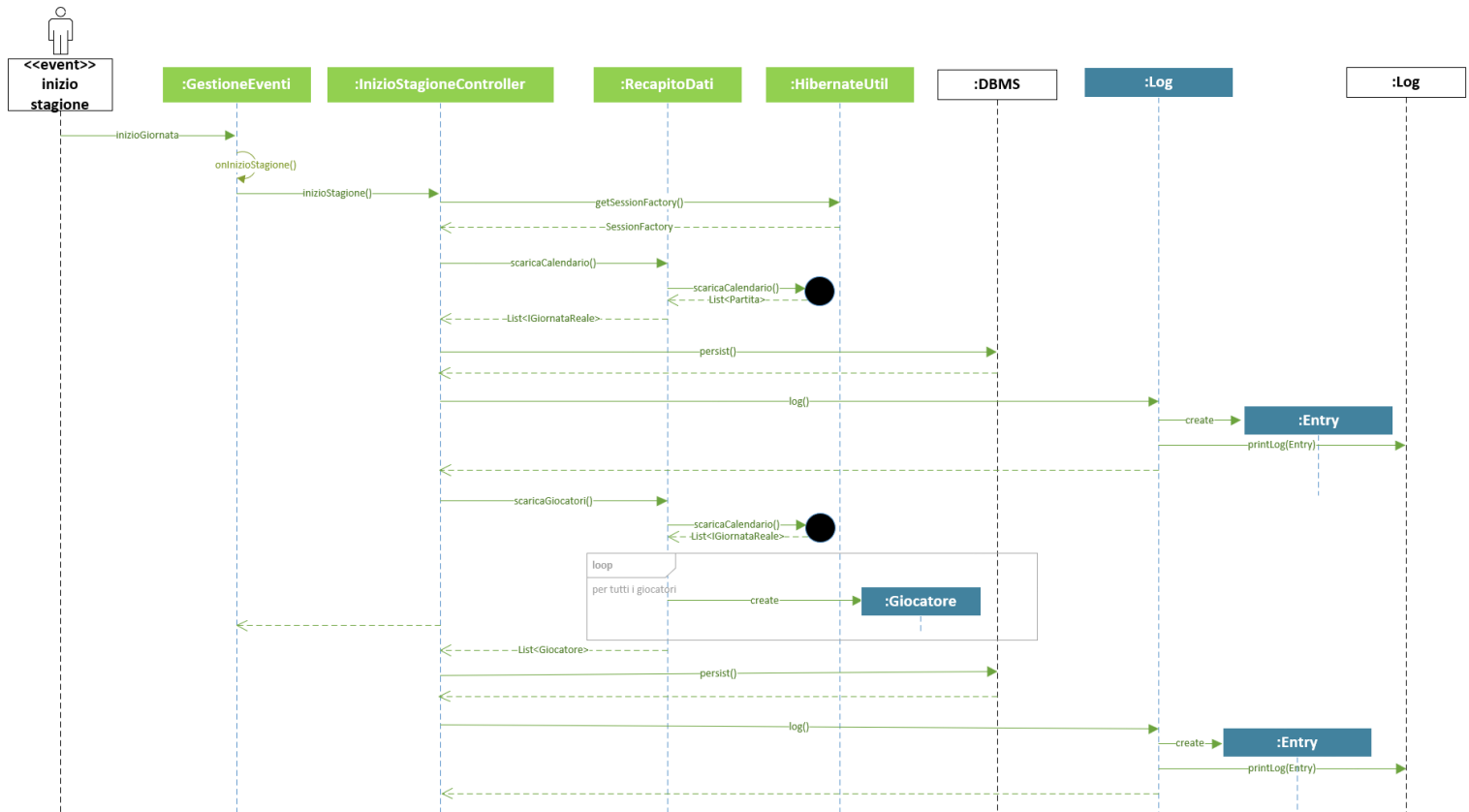
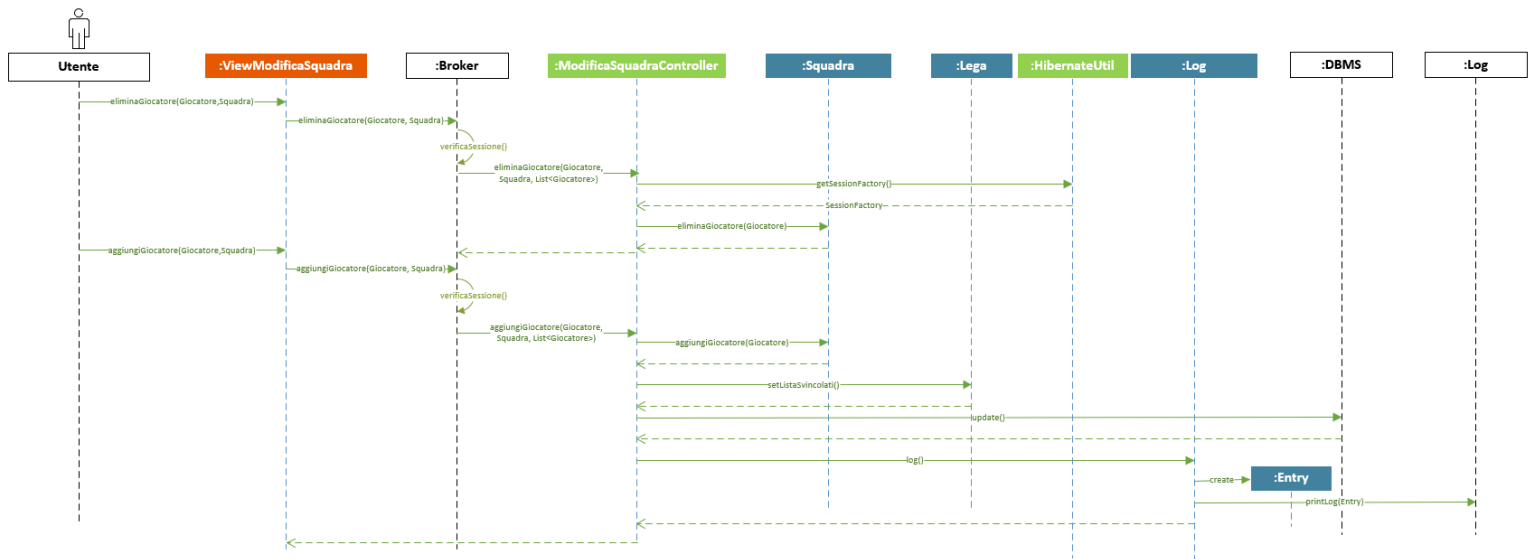


Diagramma di Sequenza: Inizio stagione



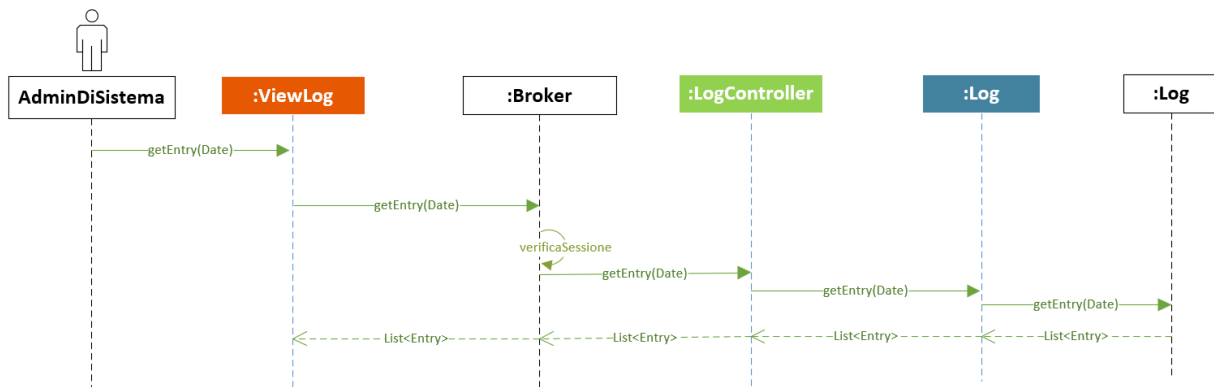
All'inizio della stagione l'applicazione è interessata a scaricare dal sistema esterno il calendario del campionato LBA e la lista dei giocatori di tutte le squadre.

Diagramma di Sequenza: Modifica squadra



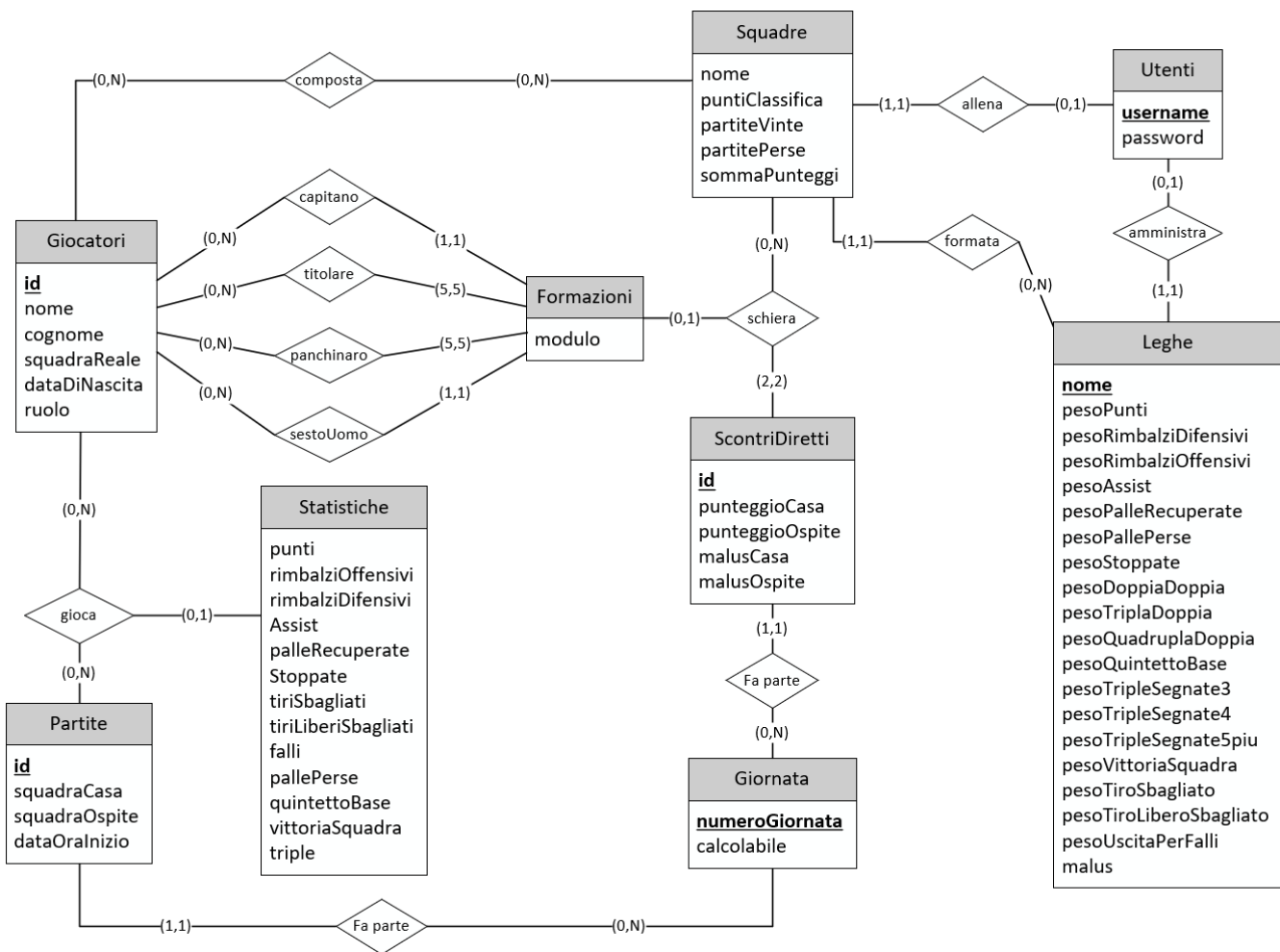
La procedura di modifica squadra rappresenta un'operazione critica per vari motivi e per questo necessita di accurati controlli da parte del sistema. Il numero di invocazioni di `eliminaGiocatore` deve essere uguale al numero di invocazioni di `aggiungiGiocatore` in modo da non lasciare una squadra non completa. Inoltre, `aggiungiGiocatore` deve controllare che la squadra non sia già completa, e che il ruolo del giocatore che si vuole aggiungere rispetti i vincoli della squadra.

Diagramma di Sequenza: Visualizzazione Log



PROGETTAZIONE DELLA PERSISTENZA

DIAGRAMMA E-R



L'entità Squadre è identificata mediante un identificatore esterno: il nome della Squadra è univoco solamente all'interno della propria lega.

Anche Statistiche e Formazioni sono identificate da identificatori esterni, in particolare un'istanza Statistiche è identificata dalla coppia Giocatori-Partite con cui è in relazione. Allo stesso modo anche un'istanza di Formazioni è identificata univocamente dalla coppia Squadre-ScontriDiretti con cui è in relazione.

La lista svincolati di una lega non viene memorizzata esplicitamente, poiché tale informazione si può ricavare considerando le istanze di Giocatori che non sono associate a nessuna squadra della lega.

Si evidenziano i seguenti vincoli non rappresentabili dal diagramma:

- Un utente può essere amministratore solamente di una lega in cui allena una squadra
- La formazione di una squadra in uno scontro diretto è composta solamente da giocatori che in quel momento facevano parte di quella squadra

Nell'entità Utente è stato inserito l'attributo password, non presente nel modello del dominio. Per ragioni di sicurezza le password degli utenti sono memorizzate nel DB sotto forma di hash calcolato combinando la password con un salt. Le password rispettano il formato:

```
$id_algoritmo_di_hash$salt$fingerprint_calcolato_sul_salt_concatenato_alla_password
```

FORMATO DEI FILE DI LOG

I log dell'applicazione vengono salvati in file di testo con il seguente formato:

DataOra – Operazione – Esecutore

PROGETTAZIONE DEL COLLAUDO

```
public class CreazioneCampionatoTest {

    @Test
    public void testCreazioneCampionato() {
        ICreazioneCampionato c=new CreazioneCampionatoController();
        IVistaCampionato v=new VistaCampionatoController();

        List<Giocatore> giocatori1=new ArrayList<>();
        Giocatore g1=new Giocatore("Abass", "Abass", "Virtus Bologna", LocalDate.of(1993, 1, 27), Ruolo.ALA);
        giocatori1.add(g1);
        Giocatore g2=new Giocatore("Nicola", "Akele", "Brescia", LocalDate.of(1995, 11, 7 ), Ruolo.ALA);
        giocatori1.add(g2);
        Giocatore g3=new Giocatore("Jamal", "Jones", "Dinamo Sassari", LocalDate.of(1993, 2, 17), Ruolo.ALA);
        giocatori1.add(g3);
        Giocatore g4=new Giocatore("Kaspar", "Treier", "Dinamo Sassari", LocalDate.of(1999, 9, 19), Ruolo.ALA);
        giocatori1.add(g4);
        Giocatore g5=new Giocatore("Stefano", "Gentile", "Dinamo Sassari", LocalDate.of(1989, 9, 20), Ruolo.ALA);
        giocatori1.add(g5);
        Giocatore g6=new Giocatore("Timothe", "Luwawu-Cabarrot", "EA7 Milano", LocalDate.of(1995, 5, 9), Ruolo.GUARDIA);
        giocatori1.add(g6);
        Giocatore g7=new Giocatore("Nazareth", "Mitrou-Long", "EA7 Milano", LocalDate.of(1993, 8, 3), Ruolo.GUARDIA);
        giocatori1.add(g7);
        Giocatore g8=new Giocatore("Stefano", "Tonut", "EA7 Milano", LocalDate.of(1993, 11, 7), Ruolo.GUARDIA);
        giocatori1.add(g8);
        Giocatore g9=new Giocatore("Shabazz", "Napier", "EA7 Milano", LocalDate.of(1991, 7, 14), Ruolo.GUARDIA);
        giocatori1.add(g9);
        Giocatore g10=new Giocatore("Tommaso", "Baldasso", "EA7 Milano", LocalDate.of(1998, 1, 29), Ruolo.GUARDIA);
        giocatori1.add(g10);
        Giocatore g11=new Giocatore("Kyle", "Hines", "EA7 Milano", LocalDate.of(1986, 9, 2), Ruolo.CENTRO);
        giocatori1.add(g11);
        Giocatore g12=new Giocatore("Paul Stephan", "Biligha", "EA7 Milano", LocalDate.of(1990, 5, 31), Ruolo.CENTRO);
        giocatori1.add(g12);
        Giocatore a13=new Giocatore("Giampaolo", "Ricci", "EA7 Milano", LocalDate.of(1991, 9, 27), Ruolo.CENTRO);
        giocatori1.add(a13);

        List<Giocatore> giocatori2=new ArrayList<>();
        Giocatore a1=new Giocatore("Luigi", "Datome", "EA7 Milano", LocalDate.of(1987, 11, 27), Ruolo.ALA);
        giocatori2.add(a1);
        Giocatore a2=new Giocatore("Leo", "Menalo", "Virtus Bologna", LocalDate.of(2002, 1, 6 ), Ruolo.ALA);
        giocatori2.add(a2);
        Giocatore a3=new Giocatore("Kyle", "Weems", "Virtus Bologna", LocalDate.of(1989, 8, 23), Ruolo.ALA);
        giocatori2.add(a3);
        Giocatore a4=new Giocatore("Semi", "Ojeleye", "Virtus Bologna", LocalDate.of(1994, 12, 5), Ruolo.ALA);
        giocatori2.add(a4);
        Giocatore a5=new Giocatore("Stefano", "Gentile", "Dinamo Sassari", LocalDate.of(1989, 9, 20), Ruolo.ALA);
        giocatori2.add(a5);
        Giocatore a6=new Giocatore("Milos", "Teodosic", "Virtus Bologna", LocalDate.of(1987, 3, 19), Ruolo.GUARDIA);
        giocatori2.add(a6);
        Giocatore a7=new Giocatore("Daniel", "hackett", "Virtus Bologna", LocalDate.of(1987, 12, 19), Ruolo.GUARDIA);
        giocatori2.add(a7);
        Giocatore a8=new Giocatore("niccolo", "Mannion", "Virtus Bologna", LocalDate.of(2001, 3, 14), Ruolo.GUARDIA);
        giocatori2.add(a8);
        Giocatore a9=new Giocatore("Giovanni", "Faldini", "Virtus Bologna", LocalDate.of(2005, 4, 18), Ruolo.GUARDIA);
        giocatori2.add(a9);
        Giocatore a10=new Giocatore("Marco", "Belinelli", "Virtus Bologna", LocalDate.of(1986, 3, 25), Ruolo.GUARDIA);
        giocatori2.add(a10);
        Giocatore a11=new Giocatore("Tornike", "Shengelia", "Virtus Bologna", LocalDate.of(1991, 10, 5), Ruolo.CENTRO);
        giocatori2.add(a11);
        Giocatore a12=new Giocatore("Amedeo", "Tessitori", "Reyer Venezia", LocalDate.of(1994, 10, 7), Ruolo.CENTRO);
        giocatori2.add(a12);
        Giocatore a13=new Giocatore("Jacorey", "Williams", "Napoli", LocalDate.of(1994, 6, 12), Ruolo.CENTRO);
        giocatori2.add(a13);
    }
}
```



```

//ci creiamo l'utente amministratore del campionato
Utente amministratore=new Utente("amministratore");
Utente utente=new Utente("utente 1");
String nomeLega="fantaBasket";
c.creaCampionato(amministratore, nomeLega);

//otteniamo dal db la lega creata dal metodo creaCampionato()
Lega lega;
HibernateUtil h = new HibernateUtil();
Session session;
Transaction tx;
try {
    session = h.getSessionFactory().openSession();
    tx = session.beginTransaction();
    Query query = session.createQuery("from " + Lega.class.getSimpleName() + " where nome = ?");
    query.setString(0, nomeLega);
    lega = query.uniqueResult();
} catch (Exception e) {
    if (tx != null) {
        try {
            tx.rollback();
        } catch (Exception e2) {
            e2.printStackTrace();
        }
    }
    e.printStackTrace();
} finally {
    if (session != null) {
        session.close();
    }
}

assertEquals(amministratore, lega.getAmministratore());
assertEquals(nomeLega, lega.getNomelega());

c.inserisciSquadra(amministratore, "squadraAmministratore");
c.inserisciSquadra(utente, "squadraUtente");

assertEquals(amministratore, v.getSquadra(lega, "squadraAmministratore").getAllenatore());
assertEquals("squadraAmministratore", v.getSquadra(lega, "squadraAmministratore").getNome());
assertEquals(utente, v.getSquadra(lega, "squadraUtente").getAllenatore());
assertEquals("squadraUtente", v.getSquadra(lega, "squadraUtente").getNome());

c.inserisciGiocatori(v.getSquadra(lega, "squadraAmministratore"), giocatori1);
c.inserisciGiocatori(v.getSquadra(lega, "squadraUtente"), giocatori2);

assertEquals(giocatori1, v.getSquadra(lega, "squadraAmministratore").getGiocatori());
assertEquals(giocatori2, v.getSquadra(lega, "squadraUtente").getGiocatori());

PesiStatistiche pesi=new PesiStatistiche();//costruttore senza parametri che mette tutti i valori a quelli di default
c.inserisciPesiStatistiche(lega, pesi);

assertEquals(pesi, lega.getPesiStatistiche());

//controlliamo la modifica dei pesi delle statistiche
IPesiStatistiche p=new PesiStatisticheController();
pesi.setPesoPunti(5);
p.modificaPesiStatistiche(pesi,lega);
assertEquals(pesi, lega.getPesiStatistiche());

//controlliamo la modifica di una squadra mediante il controller modificaSquadraController
IModificaSquadra m=new modificaSquadraController();

Giocatore b1=new Giocatore("Andrea", "Cinciarini", "Reggio Emilia", LocalDate.of(1986, 11, 27));
Giocatore c1=new Giocatore("Marcus", "Lee", "Reggio Emilia", LocalDate.of(1994, 9, 14));

```

```

List<Giocatore> squadraPrimaDelleModifiche=new ArrayList<>(v.getSquadra(lega, "squadraAmministratore").getGiocatori());
m.aggiungiGiocatore(b1,v.getSquadra(lega, "squadraAmministratore"), lega.getSvincolati());
//devono essere uguali perchè la squadra è già al completo
assertEquals(squadraPrimaDelleModifiche, v.getSquadra(lega, "squadraAmministratore").getGiocatori());
//quindi non si possono aggiungere dei giocatori

m.eliminaGiocatore(g8, v.getSquadra(lega, "squadraAmministratore"), lega.getSvincolati());
giocatori1.remove(g8);

//abbiamo rimosso una guardia, quindi possiamo inserire solo una guardia

//proviamo ad inserire un centro
squadraPrimaDelleModifiche=new ArrayList<>(v.getSquadra(lega, "squadraAmministratore"));
m.aggiungiGiocatore(c1,v.getSquadra(lega, "squadraAmministratore"), lega.getSvincolati());
assertEquals(squadraPrimaDelleModifiche, v.getSquadra(lega, "squadraAmministratore").getGiocatori());

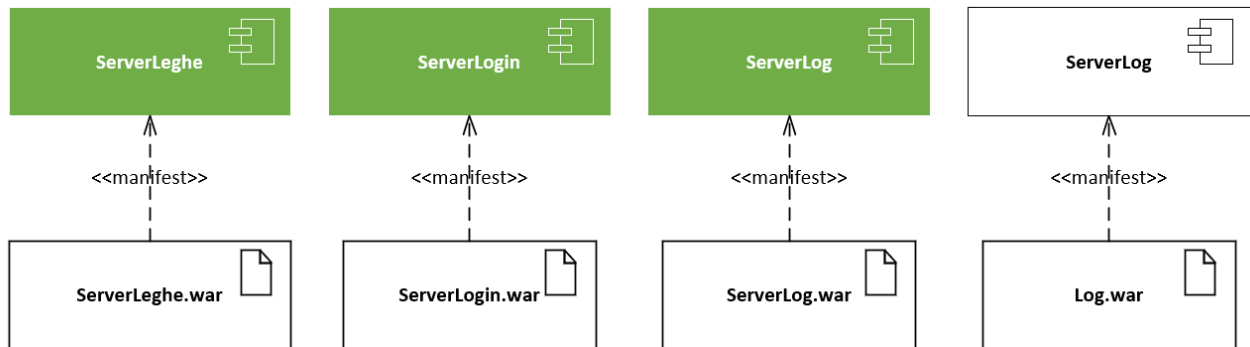
//inseriamo il giocatore giusto
giocatori1.add(b1);
m.aggiungiGiocatore(b1,v.getSquadra(lega, "squadraAmministratore"), lega.getSvincolati());
assertEquals(giocatori1, v.getSquadra(lega, "squadraAmministratore").getGiocatori());
}

@Test
public void TestRegistrazioneLogin() {
    ILogin login = new LoginController();
    IRegistrazione registrazione = new RegistrazioneController();
    registrazione.registraUtente("myUsername", "myPassword");
    assertEquals("utente", login.verificaCredenziali("myUsername", "myPassword"));
}
}

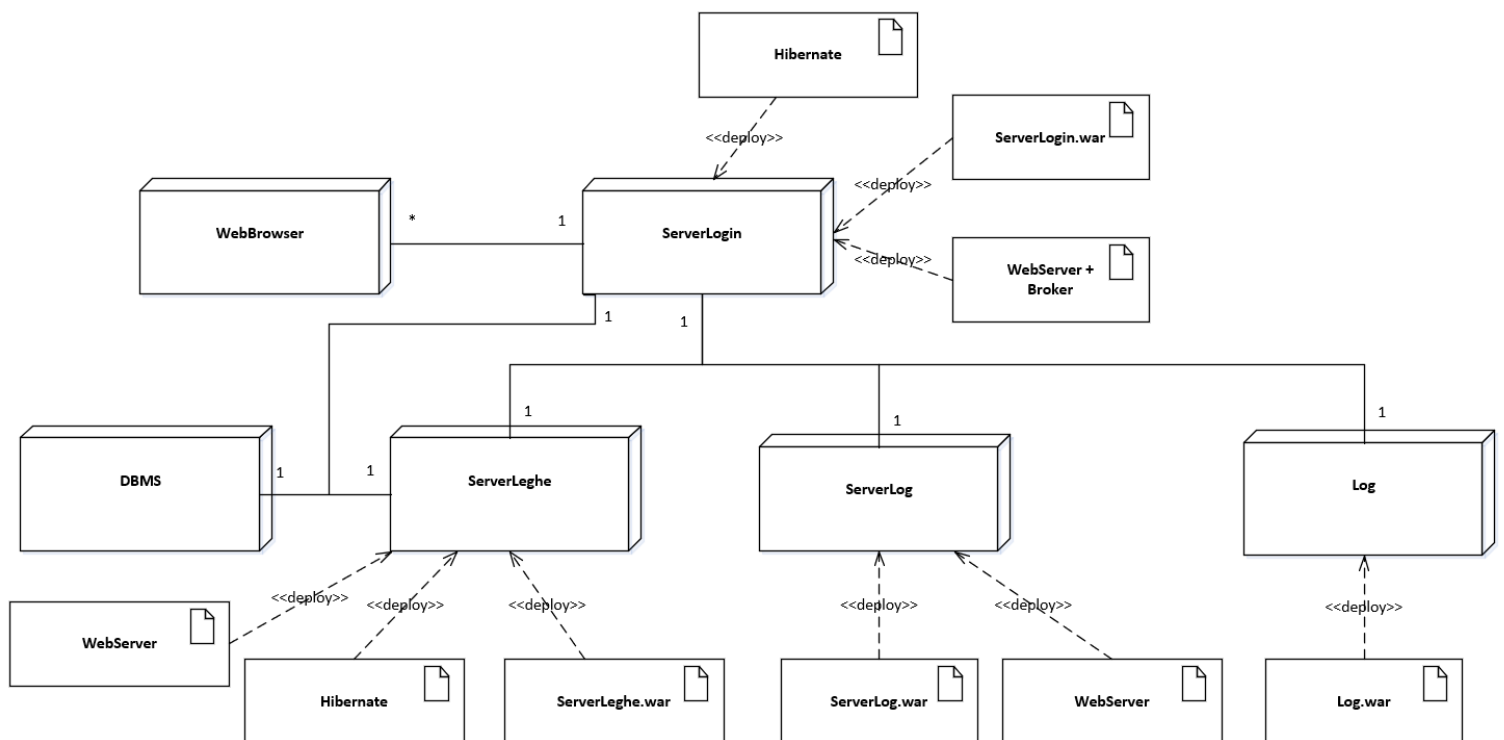
```

DEPLOYMENT

Artefatti



Deployment Type-Level



Per implementare il pattern Broker si sfrutta quello messo a disposizione dal web server installato sul ServerLogin.